Filière de Big Data et Intelligence Artificielle
École Nationale des Sciences Appliquées de Tétouan

Plateforme de détection d'anomalies réseau et d'analyse de menaces (Threat Intelligence) en temps réel.



Groupe :
SOULAMA Doubie Judicael Noé
EL ALAOUI SALEH Zouhair
PROBI Nouhaila
EL JABRI Hafsa
SIMBRE Achraf

Encadré par :
Pr. SASSI Imad

2025-2026

Résumé

Ce rapport présente la conception et l'implémentation d'une plateforme SIEM (*Security Information and Event Management*) temps réel reposant sur un écosystème Big Data. Les flux réseau sont ingérés via Apache Kafka, traités par Spark Structured Streaming, puis soumis à quatre modèles d'intelligence artificielle — Random Forest, Isolation Forest, Réseau de Neurones MLP et XGBoost — combinés par un mécanisme de vote majoritaire pour la détection d'intrusions. Entraînés sur le dataset UNSW-NB15, ces modèles atteignent un F1-score jusqu'à 0,9165 et un AUC-ROC de 0,9836 pour le meilleur d'entre eux (XGBoost), avec un temps d'inférence de 0,0029 ms par échantillon adapté au traitement en continu. Les alertes générées sont indexées dans Elasticsearch et visualisées via un dashboard Kibana, tandis que les logs bruts sont archivés dans MongoDB. L'association d'un modèle non supervisé (Isolation Forest) aux modèles supervisés permet en outre de couvrir les attaques de type *zero-day* non présentes dans les données d'entraînement.

Mots-clés : SIEM, Big Data, Apache Kafka, Spark Streaming, Machine Learning, Détection d'Intrusions, Vote Majoritaire, UNSW-NB15, Elasticsearch, Kibana.

Table des matières

Résumé **1**

1 Introduction **7**
1.1 Contexte . . . 7
1.2 Problématique . . . 7
1.3 Objectifs du projet . . . 7
1.4 Structure du rapport . . . 8

2 Architecture du Système **9**
2.1 Vue globale — schéma du pipeline . . . 9
2.2 Choix de l'architecture streaming . . . 10
2.3 Description des composants . . . 10
2.3.1 Apache Kafka . . . 10
2.3.2 Apache Spark . . . 11
2.3.3 MongoDB . . . 11
2.3.4 Elasticsearch . . . 11
2.3.5 Kibana . . . 11
2.4 Infrastructure — Docker Compose . . . 12

3 Données Utilisées **13**
3.1 Dataset UNSW-NB15 . . . 13
3.1.1 Présentation et origine . . . 13
3.1.2 Structure et features . . . 13
3.2 Logs SSH OpenSSH (2 000 lignes) . . . 14
3.3 Données synthétiques pour la démo live . . . 14
3.4 Qualité des données . . . 14
3.4.1 Complétude et valeurs manquantes . . . 14
3.4.2 Déséquilibre des classes . . . 15
3.4.3 Encodage et normalisation . . . 15

4 Ingestion avec Apache Kafka **17**
4.1 Architecture Kafka — topics, producers, consumers . . . 17
4.2 Script `producer.py` . . . 17
4.3 Résultats d'ingestion . . . 18
4.3.1 Topic `logs-systeme` — 2 000 messages . . . 18
4.3.2 Topic `flux-reseau` — 82 332 messages . . . 18
4.3.3 Topic `threat-intel` . . . 18

5 Modélisation IA - Étude Comparative **19**
5.1 Préparation du dataset UNSW-NB15 . . . 19
5.2 Prétraitement des données . . . 19
5.2.1 Encodage des variables catégorielles (LabelEncoder) . . . 19
5.2.2 Normalisation avec RobustScaler . . . 20
5.2.3 Suppression des variables à variance nulle . . . 20
5.2.4 Traitement des valeurs extrêmes (clip ±10) . . . 20
5.3 Modèle 1 - Random Forest (supervisé) . . . 20
5.4 Modèle 2 - Isolation Forest (non supervisé) . . . 21
5.5 Modèle 3 — Réseau de Neurones MLP (supervisé) . . . 21
5.6 Modèle 4 - XGBoost (supervisé) . . . 22
5.7 Tableau comparatif . . . 22
5.7.1 Temps d'inférence . . . 23
5.8 Vote majoritaire et intégration dans le pipeline . . . 23
5.8.1 Principe du vote majoritaire . . . 23
5.8.2 Architecture du pipeline Spark . . . 24
5.8.3 Structure d'un document Elasticsearch . . . 24
5.9 Analyse et recommandation . . . 25

6 Traitement avec Spark Streaming **26**
6.1 Test sur machine locale . . . 26
6.1.1 Architecture Spark Structured Streaming . . . 26
6.1.2 Lecture des flux depuis Kafka . . . 26
6.1.3 Prétraitement en streaming . . . 27
6.1.4 Chargement des modèles sauvegardés . . . 28
6.1.5 Inférence sur les micro-batchs . . . 28
6.1.6 Vote majoritaire . . . 29
6.1.7 Attribution du niveau de sévérité . . . 30
6.1.8 Stockage des résultats . . . 30
6.1.9 Calcul des statistiques par fenêtres temporelles . . . 31
6.1.10 Exécution continue du pipeline . . . 33
6.2 Test en architecture distribuée (2 machines) . . . 33
6.2.1 Choix de l'architecture et comparaison . . . 33
6.2.2 Schéma du déploiement distribué . . . 34
6.2.3 Configuration réseau et déploiement . . . 35

7 Stockage NoSQL **36**
7.1 MongoDB . . . 36
7.1.1 Collection `logs_bruts` . . . 36
7.1.2 Collection `alertes` . . . 37
7.1.3 Collection `stats_sessions` . . . 37
7.1.4 Index et optimisation . . . 37
7.2 Elasticsearch . . . 37
7.2.1 Index `siem-alertes` . . . 38
7.2.2 Mapping des champs . . . 38
7.2.3 Insertion en bulk . . . 38
7.3 Contrôle de qualité des données . . . 39
7.3.1 Valeurs manquantes . . . 39
7.3.2 Doublons . . . 39
7.3.3 Déséquilibre des classes . . . 39

7.3.4 Valeurs infinies et variance nulle 39

8 Visualisation et Dashboards Kibana **41**
8.1 Configuration des index patterns 41
8.2 Dashboard des alertes de sécurité 41
8.3 Dashboard des statistiques temporelles 42
8.4 Analyse visuelle en temps réel 42

9 Éthique et Conformité **43**
9.1 Protection des données — RGPD 43
9.2 Conformité légale 43

10 Conclusion et Perspectives **45**
10.1 Synthèse des résultats 45
10.2 Limites du système 45
10.3 Perspectives d'amélioration 46

A Code Source Complet **48**

B Captures d'écran Kibana **49**

C Configuration Docker Compose **54**

Liste des tableaux

2.1 Services Docker Compose du projet SIEM 12

3.1 Statistiques descriptives des features clés — jeu d'entraînement UNSW-NB15 15
3.2 Distribution des classes dans les jeux d'entraînement et de test 15

5.1 Comparaison des modèles sur le test set UNSW-NB15 22
5.2 Temps d'inférence par échantillon sur le test set 23
5.3 Échelle de sévérité selon le vote 23

6.1 Structure de stockage des modèles de détection 28
6.2 Type de sortie et score de confiance pour chaque modèle 29
6.3 Principe du vote majoritaire et du calcul du score de confiance 29
6.4 Attribution du niveau de sévérité selon le nombre de votes 30
6.5 Résumé console des informations affichées pour chaque batch 30
6.6 Principaux champs stockés dans la collection `logs_bruts` 31
6.7 Structure du mapping Elasticsearch pour les alertes SIEM 31
6.8 Principaux indicateurs calculés pour chaque fenêtre glissante 32
6.9 Caractéristiques du pipeline tumbling 32
6.10 Gestion de l'exécution continue et de l'arrêt propre du pipeline 33

7.1 Index MongoDB créés sur la collection `logs_bruts` 37
7.2 Mapping des champs de l'index `alertes-securite` 38

C.1 Récapitulatif des services Docker Compose 54

Table des figures

2.1 Vue globale du pipeline SIEM Big Data 10

6.1 Comparaison entre architecture mono-machine et cluster distribué 34
6.2 Schéma du pipeline distribué : Machine 1 (producteur) et Machine 2 (traitement IA & visualisation) 35

B.1 Répartition des protocoles réseau — graphique en anneau (UDP 42,96 %, TCP 41,35 %) 49
B.2 Distribution des types d'attaques — Generic 33,69 %, Normal 20,02 %, Exploits 19,84 % 50
B.3 Nombre de flux par protocole — barres verticales (Top 5) 50
B.4 Métriques moyennes par protocole — octets source/destination, durée et taux de transfert 51
B.5 Attaques par protocole et catégorie — barres empilées 51
B.6 Score de confiance médian des prédictions (*Median of confidence*) 52
B.7 Dernière valeur du score d'anomalie Isolation Forest (`if_score` $\approx 0{,}4$) . . . 52
B.8 Évolution temporelle de la somme des votes et du nombre de flux — fenêtres de 5 minutes 52
B.9 Évolution temporelle des flux par catégorie d'attaque — fenêtres de 5 minutes 53
B.10 Distribution des votes ATTACK par flux — majorité à 4/4 modèles ($\approx$48 000 enregistrements) 53

Chapitre 1

Introduction

1.1 Contexte

La cybersécurité constitue aujourd'hui un enjeu stratégique majeur pour les organisations, face à une menace informatique en constante évolution. Les systèmes d'information modernes génèrent des volumes croissants de logs et d'événements réseau, rendant impossible toute surveillance manuelle exhaustive. Dans ce contexte, les plateformes SIEM (*Security Information and Event Management*) jouent un rôle central en centralisant, corrélant et analysant automatiquement ces flux d'événements pour détecter les comportements malveillants.

Les technologies Big Data offrent aujourd'hui des capacités de traitement en temps réel à grande échelle, ouvrant la voie à des architectures SIEM plus performantes, capables d'absorber des millions d'événements par seconde tout en maintenant une latence de détection minimale.

1.2 Problématique

Les solutions de détection d'intrusions traditionnelles reposent souvent sur des règles statiques ou des signatures connues, ce qui les rend inefficaces face aux attaques zero-day et aux comportements anormaux subtils. Par ailleurs, la volumétrie croissante du trafic réseau dépasse les capacités des approches batch classiques, qui introduisent des délais incompatibles avec une réponse aux incidents en temps réel.

Ce projet répond à la problématique suivante : *comment concevoir une plateforme de détection d'anomalies réseau capable de traiter des flux continus à haute fréquence, en combinant plusieurs modèles d'intelligence artificielle pour une détection robuste et en temps réel des intrusions ?*

1.3 Objectifs du projet

Les objectifs principaux de ce projet sont les suivants :

— Mettre en place un pipeline de streaming Big Data complet, de l'ingestion à la visualisation, reposant sur Apache Kafka et Spark Structured Streaming.

— Entraîner et comparer quatre modèles d'apprentissage automatique — Random Forest, Isolation Forest, Réseau de Neurones MLP et XGBoost — sur le dataset de référence UNSW-NB15.

— Combiner les prédictions des modèles via un mécanisme de vote majoritaire pour améliorer la robustesse de la détection et réduire les faux positifs.

— Stocker les alertes générées dans Elasticsearch et les logs bruts dans MongoDB, pour une persistance opérationnelle et forensique.
— Visualiser en temps réel l'activité réseau et les alertes de sécurité via des tableaux de bord Kibana interactifs.
— Assurer la conformité éthique et réglementaire du projet, notamment au regard du RGPD et de la loi marocaine 09-08.

1.4 Structure du rapport

Ce rapport est organisé comme suit. Le chapitre 2 présente l'architecture globale du système et les choix technologiques retenus. Le chapitre 3 décrit les datasets utilisés ainsi que la démarche de contrôle qualité des données. Le chapitre 4 détaille le pipeline d'ingestion Kafka. Le chapitre 5 présente l'étude comparative des quatre modèles d'intelligence artificielle et leur intégration dans le pipeline via le vote majoritaire. Le chapitre 6 décrit le traitement en temps réel avec Spark Structured Streaming. Le chapitre 7 couvre le stockage NoSQL avec MongoDB et Elasticsearch. Le chapitre 8 présente les dashboards Kibana. Enfin, le chapitre 9 aborde les aspects éthiques et réglementaires, avant la conclusion et les perspectives du chapitre 10.

Chapitre 2

Architecture du Système

Ce chapitre présente l'architecture globale du système SIEM (*Security Information and Event Management*) développé dans le cadre de ce projet. L'architecture repose sur un pipeline de traitement en flux continu (*streaming*), conçu pour ingérer, analyser et stocker des événements réseau en temps quasi-réel, en s'appuyant sur des technologies Big Data éprouvées et déployées via Docker Compose.

2.1 Vue globale — schéma du pipeline

Le pipeline de traitement suit un flux de données linéaire et découplé, organisé en quatre grandes phases :

1. **Ingestion** : un producteur Kafka lit les enregistrements du dataset UNSW-NB15 et les publie sur un topic dédié (`flux-reseau`) à un rythme configurable, simulant un flux réseau en temps réel.
2. **Traitement** : Apache Spark Structured Streaming consomme les messages Kafka, applique le prétraitement des données (encodage, normalisation, fenêtrage temporel) puis sollicite les modèles de machine learning chargés en mémoire pour obtenir une prédiction par événement.
3. **Stockage** : les logs bruts sont persistés dans MongoDB (`logs_bruts`) tandis que les alertes enrichies sont indexées dans Elasticsearch (`alertes-securite`) pour permettre une recherche et une visualisation rapides.
4. **Visualisation** : Kibana expose les données Elasticsearch sous forme de tableaux de bord interactifs, offrant une vue en temps réel de l'activité réseau et des alertes de sécurité.

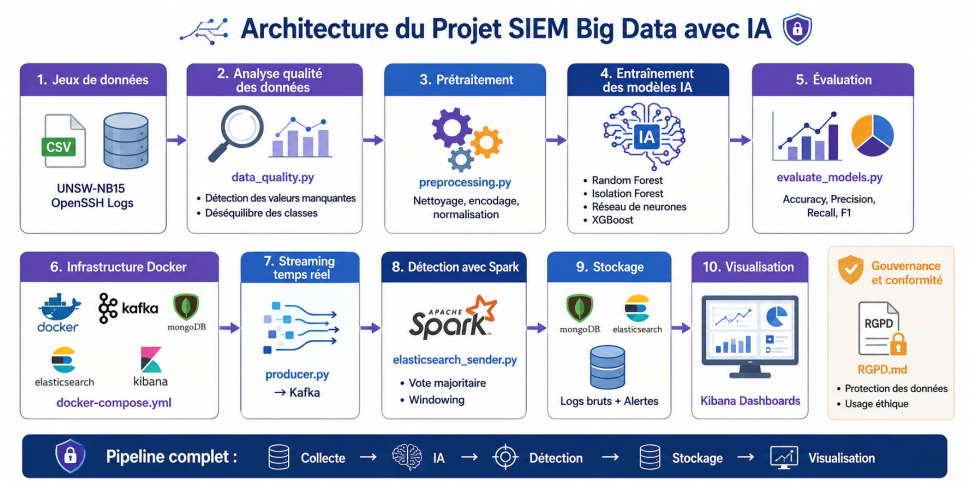


FIGURE 2.1 – Vue globale du pipeline SIEM Big Data

2.2 Choix de l'architecture streaming

Le choix d'une architecture orientée *streaming* plutôt que *batch* est motivé par plusieurs contraintes inhérentes au domaine de la cybersécurité :

- **Latence de détection** : une architecture batch introduirait un délai d'analyse incompatible avec la nécessité de détecter et d'alerter en temps réel sur des attaques en cours.
- **Volume continu** : le trafic réseau est par nature un flux continu et non borné (*unbounded stream*), que les frameworks de batch processing ne savent pas traiter nativement.
- **Fenêtrage temporel** : l'analyse comportementale (détection d'anomalies sur des fenêtres glissantes) nécessite un moteur capable d'agréger dynamiquement des événements sur des intervalles de temps définis, fonctionnalité native dans Spark Structured Streaming.
- **Découplage producteur/consommateur** : l'interposition de Kafka entre la source de données et le moteur de traitement garantit la résilience du pipeline face aux pics de charge ou aux indisponibilités temporaires d'un composant.

Cette architecture s'inscrit dans le modèle **Lambda** simplifié, où la couche *speed layer* (Spark Streaming) assure le traitement temps réel, et la couche *serving layer* (MongoDB + Elasticsearch) permet l'interrogation et l'archivage des résultats.

2.3 Description des composants

Le système est composé de cinq composants technologiques principaux, chacun jouant un rôle précis dans le pipeline.

2.3.1 Apache Kafka

Apache Kafka est la colonne vertébrale du système de messagerie. Il joue le rôle de *message broker* distribué, découplant temporellement et logiquement le producteur de données (le simulateur de trafic réseau) des consommateurs (Spark Streaming).

Dans ce projet, un unique topic `flux-reseau` est utilisé. Le producteur (`producer.py`) lit séquentiellement les enregistrements du dataset UNSW-NB15 et les sérialise en JSON avant de les publier sur ce topic. Kafka garantit la durabilité des messages et permet

à Spark de rejouer les événements en cas de défaillance, grâce à la gestion des *offsets* consommateurs.

Paramètres de configuration :

- Broker : `localhost:9092`
- Topic : `flux-reseau`
- Format de sérialisation : JSON (UTF-8)

2.3.2 Apache Spark

Apache Spark est le moteur de traitement distribué au cœur du pipeline. Il est utilisé en mode **Structured Streaming**, qui permet d'exprimer les transformations sur les flux de données avec la même API DataFrame que pour le traitement batch, tout en gérant nativement l'état, le fenêtrage et la tolérance aux pannes via les *checkpoints*.

Le pipeline Spark réalise successivement les opérations suivantes :

1. Lecture des messages depuis le topic Kafka et désérialisation JSON ;
2. Prétraitement : encodage `LabelEncoder` des colonnes catégorielles (`proto`, `service`, `state`) et normalisation `RobustScaler` des features numériques ;
3. Agrégation par fenêtres temporelles glissantes (*windowing*) ;
4. Inférence par les les quatre modèles ML (Random Forest, Isolation Forest, réseau de neurones) et calcul du vote majoritaire ;
5. Envoi des alertes vers Elasticsearch et archivage dans MongoDB.

2.3.3 MongoDB

MongoDB est le système de base de données NoSQL orienté documents utilisé pour la persistance brute et opérationnelle des données. Il stocke trois types d'informations : les logs réseau bruts complets (`logs_bruts`), les alertes de sécurité en copie de sauvegarde (`alertes`), et les statistiques de session agrégées (`stats_sessions`).

Son schéma flexible (sans structure fixe imposée) le rend particulièrement adapté à l'ingestion de logs hétérogènes, où le nombre et la nature des champs peuvent varier d'une source à l'autre. Pour une description détaillée des collections et index, se référer au chapitre 7 (*Stockage NoSQL*).

2.3.4 Elasticsearch

Elasticsearch est le moteur d'indexation et de recherche distribué utilisé pour stocker et interroger les alertes de sécurité. Son architecture orientée documents JSON et ses capacités de recherche plein texte, de filtrage sur des champs typés et d'agrégation en temps réel en font le complément naturel de Kibana pour la visualisation SIEM.

L'index `alertes-securite` est configuré avec un mapping explicite définissant 21 champs (adresses IP, protocoles, scores de prédiction, sévérité, etc.), garantissant la cohérence des types lors des agrégations et des visualisations. Les alertes sont insérées en mode *bulk* pour maximiser le débit d'indexation.

2.3.5 Kibana

Kibana est l'interface de visualisation native d'Elasticsearch. Dans le contexte de ce SIEM, il offre :

— Des **tableaux de bord** (*dashboards*) affichant en temps réel le volume d'alertes, leur répartition par sévérité, protocole ou catégorie d'attaque ;
— Des **graphiques temporels** permettant d'identifier des pics d'activité malveillante sur des fenêtres de temps définies ;
— Un **explorateur de données** (*Discover*) pour l'investigation manuelle d'alertes spécifiques ;
— Des **filtres interactifs** sur tous les champs indexés (adresse IP source, type d'attaque, score de confiance, etc.).

Kibana est accessible sur `http://localhost:5601` dans l'environnement Docker Compose du projet.

2.4 Infrastructure — Docker Compose

L'ensemble des composants du système est orchestré via **Docker Compose**, permettant un déploiement reproductible et isolé en un seul fichier de configuration (`docker-compose.yml`).

Le fichier définit les services suivants :

Table 2.1 – Services Docker Compose du projet SIEM

Service	Image	Port exposé	Rôle
`zookeeper`	`confluentinc/...`	2181	Coordination Kafka
`kafka`	`confluentinc/...`	9092	Message broker
`mongodb`	`mongo:latest`	27017	Base documentaire
`elasticsearch`	`elasticsearch:8.x`	9200	Moteur d'indexation
`kibana`	`kibana:8.x`	5601	Visualisation
`Spark`	`apache/spark:3.5.0`	8080	Streaming et modeles

L'utilisation de Docker Compose présente plusieurs avantages dans le cadre de ce projet :

— **Reproductibilité** : l'environnement est identique sur toute machine disposant de Docker, éliminant les problèmes de configuration locale ;
— **Isolation** : chaque service s'exécute dans son propre conteneur, évitant les conflits de dépendances ;
— **Simplicité de démarrage** : un unique `docker compose up -d` initialise l'ensemble de l'infrastructure en quelques secondes ;
— **Scalabilité future** : la configuration Docker Compose peut être migrée vers Kubernetes pour un déploiement en production multi-nœuds sans modification du code applicatif.

Chapitre 3

Données Utilisées

3.1 Dataset UNSW-NB15

3.1.1 Présentation et origine

Le dataset UNSW-NB15 [1] a été créé en 2015 par le Cyber Range Lab de l'université de New South Wales (UNSW) en Australie, sous la direction de Nour Moustafa et Jill Slay. Il a été généré à l'aide de l'outil IXIA PerfectStorm afin de produire un trafic réseau réaliste combinant des comportements normaux et neuf catégories d'attaques modernes : Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode et Worms. Ce dataset est aujourd'hui une référence dans la communauté de la détection d'intrusions, conçu pour pallier les limites des anciens benchmarks comme KDD Cup 99, dont le trafic était devenu obsolète. Il est disponible publiquement sur le portail de recherche de l'UNSW :

`https://research.unsw.edu.au/projects/unsw-nb15-dataset`

3.1.2 Structure et features

Le dataset est composé de 257 673 enregistrements pour l'entraînement et 82 332 pour le test, soit un total de 340 005 flux réseau. Chaque flux est décrit par 44 colonnes, dont 42 features, une colonne `label` (0 = NORMAL, 1 = ATTACK) et une colonne `attack_cat` indiquant la catégorie d'attaque. Les features couvrent quatre grandes familles :

- **Features de base** : caractéristiques fondamentales du flux (`dur`, `proto`, `service`, `state`, `spkts`, `dpkts`, `sbytes`, `dbytes`, `rate`, etc.).
- **Features de contenu** : informations extraites de la couche applicative (`trans_depth`, `response_body_len`, `ct_flw_http_mthd`, `is_ftp_login`, etc.).
- **Features temporelles** : statistiques de délai et de gigue (`sinpkt`, `dinpkt`, `sjit`, `djit`, `tcprtt`, `synack`, `ackdat`).
- **Features de connexion** : compteurs de connexions récentes vers la même destination ou depuis la même source (`ct_srv_src`, `ct_dst_ltm`, `ct_src_ltm`, `ct_srv_dst`, etc.).

La distribution des classes présente un déséquilibre modéré : le jeu d'entraînement contient 56 % d'attaques et 44 % de trafic normal, tandis que le jeu de test en contient 55 % d'attaques et 45 % de normal.

3.2 Logs SSH OpenSSH (2 000 lignes)

En complément du dataset UNSW-NB15, des logs SSH générés par OpenSSH ont été intégrés au pipeline pour simuler un flux de logs système réaliste. Ces 2 000 lignes de logs contiennent des événements typiques d'un serveur SSH exposé sur Internet : tentatives de connexion réussies et échouées, attaques par force brute, scans de ports et tentatives d'authentification avec des identifiants invalides. Ces logs sont ingérés via le topic Kafka `logs-systeme` et stockés dans MongoDB pour une analyse ultérieure. Leur format texte brut est parsé par une expression régulière pour en extraire les champs pertinents : horodatage, niveau de sévérité, adresse IP source et message d'événement. Un exemple de dataset de logs SSH similaire est accessible sur Kaggle :

`https://www.kaggle.com/datasets/doragoyal/ssh-brute-force-logs`

3.3 Données synthétiques pour la démo live

Afin de démontrer le fonctionnement du pipeline en temps réel lors de la soutenance, un générateur de données synthétiques a été développé. Il produit des flux réseau au format JSON conformes au schéma UNSW-NB15, en faisant varier aléatoirement les features clés (protocole, volume d'octets, durée, TTL, etc.) autour de distributions statistiques observées dans le dataset original. Le générateur injecte ces flux dans le topic Kafka `flux-reseau` à un débit configurable, permettant de simuler une montée en charge et de visualiser en direct les alertes apparaître dans le dashboard Kibana.

3.4 Qualité des données

3.4.1 Complétude et valeurs manquantes

L'analyse de complétude du dataset UNSW-NB15 révèle une qualité exemplaire : aucune valeur manquante n'a été détectée, ni dans le jeu d'entraînement (175 341 lignes $\times$ 45 colonnes) ni dans le jeu de test (82 332 lignes $\times$ 45 colonnes). Le dataset est donc directement exploitable sans imputation préalable, ce qui simplifie significativement la phase de prétraitement.

Le contrôle des doublons confirme également l'absence totale d'entrées dupliquées dans les deux partitions (0 doublon dans le train, 0 dans le test). Aucun traitement de déduplication n'a donc été nécessaire, préservant l'intégrité statistique des ensembles d'évaluation.

L'inspection des valeurs numériques ne révèle par ailleurs aucune valeur infinie dans le dataset, évitant ainsi les problèmes de convergence pouvant survenir lors de l'entraînement de modèles basés sur des gradients. De même, aucune colonne à variance nulle n'a été identifiée parmi les 45 features, confirmant que l'ensemble des variables apporte une information discriminante potentiellement exploitable par les classifieurs.

Le tableau 3.1 présente les statistiques descriptives des features clés du jeu d'entraînement.

TABLE 3.1 – Statistiques descriptives des features clés — jeu d'entraînement UNSW-NB15

Stat	dur	sbytes	dbytes	rate	spkts	dpkts
count	175 341	175 341	175 341	175 341	175 341	175 341
mean	1,359	8 844	14 929	95 406	20,3	19,0
std	6,480	174 766	143 654	165 401	136,9	110,3
min	0,000	28	0	0,000	1	0
25%	0,000	114	0	32,79	2	0
50%	0,002	430	164	3 225,81	2	2
75%	0,668	1 418	1 102	125 000	12	10
max	60,000	12 965 230	14 655 550	1 000 000	9 616	10 974

Ces statistiques mettent en évidence une forte variabilité des features réseau (écart-type très élevé par rapport à la moyenne), caractéristique des données de trafic réseau réel. C'est précisément pour cette raison qu'un `RobustScaler` a été préféré à un `StandardScaler` pour la normalisation.

3.4.2 Déséquilibre des classes

La distribution des classes révèle un déséquilibre modéré entre les deux catégories du champ `label`, résumé dans le tableau 3.2.

TABLE 3.2 – Distribution des classes dans les jeux d'entraînement et de test

Partition	Total	Normal (0)	Attaque (1)	Ratio N/A
Entraînement	175 341	56 000 (31,9 %)	119 341 (68,1 %)	0,47 :1
Test	82 332	37 000 (44,9 %)	45 332 (55,1 %)	0,82 :1

Ce déséquilibre, bien qu'inverse à la situation classique où les attaques sont rares, a été pris en compte lors de l'entraînement de plusieurs façons :

- Paramètre `class_weight="balanced"` pour les modèles scikit-learn (Random Forest, Neural Network), qui ajuste automatiquement les poids des classes inversement proportionnellement à leur fréquence.
- Paramètre `scale_pos_weight =` $n_{\text{NORMAL}}/n_{\text{ATTACK}}$ pour XGBoost, soit un ratio de $\approx 0,47$.
- Utilisation de métriques adaptées au déséquilibre : F1-score macro, recall par classe et AUC-PR (aire sous la courbe Précision-Rappel), plus sensible au déséquilibre que l'AUC-ROC classique.

Les colonnes catégorielles présentes dans le dataset sont `proto` (133 valeurs uniques dans le train, 131 dans le test), `service` (13 valeurs uniques), `state` (9 valeurs uniques dans le train, 7 dans le test) et `attack_cat` (10 catégories d'attaques : Normal, Backdoor, Analysis, Fuzzers, Shellcode, Reconnaissance, Exploits, DoS, Worms, Generic).

3.4.3 Encodage et normalisation

Les colonnes catégorielles (`proto`, `service`, `state`) ont été encodées avec un `LabelEncoder` ajusté (*fit*) uniquement sur les données d'entraînement, puis appliqué (*transform*) au jeu

de test. Cette approche est indispensable pour éviter toute fuite d'information (*data leakage*) : le modèle ne doit pas avoir connaissance des modalités du jeu de test lors de la phase d'apprentissage. Les valeurs inconnues rencontrées dans le jeu de test (nouvelles modalités absentes du train) sont remplacées par la modalité la plus fréquente observée à l'entraînement.

La normalisation des features numériques a été réalisée avec un `RobustScaler`, dont la robustesse aux valeurs aberrantes — très fréquentes dans les données réseau (rafales de paquets, connexions anormalement longues) — en fait le choix le plus adapté au contexte SIEM. Contrairement au `StandardScaler` qui utilise la moyenne et l'écart-type, le `RobustScaler` se base sur la médiane et l'intervalle interquartile (IQR), ce qui limite fortement l'influence des valeurs extrêmes sur la mise à l'échelle. Les statistiques du tableau 3.1 confirment l'utilité de ce choix : l'écart-type du champ `rate` (165 401) est près de 1 700 fois supérieur à sa médiane (3 226).

Après normalisation, les valeurs aberrantes résiduelles sont écrêtées dans l'intervalle $[-10, +10]$ (*clipping*) afin de limiter leur impact sur les modèles les plus sensibles aux extrêmes, notamment le réseau de neurones MLP. L'ensemble de ces artefacts de prétraitement (encodeurs, scaler, paramètres de clipping) est sérialisé et sauvegardé dans le répertoire `artifacts/`, puis rechargé à froid au démarrage du pipeline Spark Streaming, évitant ainsi toute recalculation à chaque micro-batch.

Chapitre 4

Ingestion avec Apache Kafka

4.1 Architecture Kafka — topics, producers, consumers

Apache Kafka [2] est une plateforme de streaming distribuée orientée journaux (*log-based messaging*). Dans notre architecture, Kafka joue le rôle de bus de données central entre les sources de flux réseau et le moteur de traitement Spark. Trois topics ont été définis et initialisés via le script `kafka_config.py` :

- `logs-systeme` : reçoit les logs SSH bruts au format JSON, produits à partir du fichier `OpenSSH_2k_log_structured.csv` (2 000 entrées).
- `flux-reseau` : reçoit les flux réseau du dataset UNSW-NB15 (jeu de test, 82 332 enregistrements), chaque message correspondant à un flux encodé en JSON avec ses 44 features.
- `threat-intel` : réservé aux indicateurs de compromission (*Indicators of Compromise*, IoC) pour l'enrichissement Threat Intelligence des alertes.

Les trois topics sont créés automatiquement au premier envoi. Le script `kafka_config.py` envoie un message de test dans chacun pour vérifier la connectivité avec le broker (`localhost:9092`) avant le lancement du producer principal. Spark Structured Streaming joue le rôle de consumer en lisant en continu depuis `flux-reseau` pour alimenter le pipeline de détection IA.

4.2 Script `producer.py`

Le script `producer.py` assure l'ingestion des deux sources de données vers Kafka. Il utilise la bibliothèque `kafka-python` avec une sérialisation JSON de chaque enregistrement.

Listing 4.1 – Script producer.py — ingestion Kafka

```
import pandas as pd
from kafka import KafkaProducer
import json
import time

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

```

```
# 1. Logs SSH vers le topic logs-systeme
df_logs = pd.read_csv('data/OpenSSH_2k_log_structured.csv')
for _, row in df_logs.iterrows():
    producer.send('logs-systeme', value=row.to_dict())
    time.sleep(0.0001)
print(f"{len(df_logs)} logs envoyes !")

# 2. Flux reseau UNSW-NB15 vers le topic flux-reseau
df_network = pd.read_csv('data/UNSW_NB15_testing-set.csv')
for _, row in df_network.iterrows():
    producer.send('flux-reseau', value=row.to_dict())
    time.sleep(0.0001)
print(f"{len(df_network)} flux reseau envoyes !")

producer.flush()
print("Toutes les donnees ont ete envoyees vers Kafka !")
```

Chaque enregistrement est envoyé avec un délai de 0,1 ms entre deux messages (`time.sleep(0.0001)`), ce qui simule un flux continu tout en évitant de saturer le broker lors de l'ingestion initiale.

4.3 Résultats d'ingestion

4.3.1 Topic `logs-systeme` — 2 000 messages

Le fichier `OpenSSH_2k_log_structured.csv` contient 2 000 lignes de logs SSH (plus une ligne d'en-tête). Chaque message publié dans le topic `logs-systeme` est un objet JSON contenant les champs : `LineId`, `Date`, `Day`, `Time`, `Component`, `Pid`, `Content`, `EventId` et `EventTemplate`. Ces logs correspondent à des événements réels d'un serveur SSH exposé, incluant des tentatives de connexion échouées, des avertissements de sécurité et des attaques par force brute.

4.3.2 Topic `flux-reseau` — 82 332 messages

Le fichier `UNSW_NB15_testing-set.csv` contient 82 332 enregistrements (plus une ligne d'en-tête). L'intégralité du jeu de test est publiée dans le topic `flux-reseau`, soit 82 332 messages au total. Chaque message est un objet JSON de 44 champs correspondant aux features du dataset UNSW-NB15, prêt à être consommé par Spark Structured Streaming pour l'inférence en temps réel par les quatre modèles IA.

4.3.3 Topic `threat-intel`

Le topic `threat-intel` est initialisé par `kafka_config.py` avec un message de test, et reste disponible pour une extension future du système vers l'enrichissement Threat Intelligence (intégration de flux IoC externes tels que des listes d'adresses IP malveillantes ou des signatures d'attaques).

Chapitre 5

Modélisation IA - Étude Comparative

Cette section présente les quatre modèles d'apprentissage automatique entraînés sur le jeu de données UNSW-NB15 pour la détection d'intrusions réseau, ainsi que leur intégration dans le pipeline Spark via un mécanisme de vote majoritaire et l'envoi des alertes vers Elasticsearch.

5.1 Préparation du dataset UNSW-NB15

Le dataset UNSW-NB15 a été développé par l'Australian Centre for Cyber Security (ACSC). Il est composé de 175 341 entrées d'entraînement et 82 332 entrées de test, réparties sur 45 features décrivant le trafic réseau (protocole, durée, octets échangés, etc.).

Les fichiers utilisés sont :

— `UNSW_NB15_training-set.csv` — données d'entraînement

— `UNSW_NB15_testing-set.csv` — données de test

Le split train/test est celui fourni officiellement par les auteurs du dataset, garantissant la reproductibilité des expériences.

5.2 Prétraitement des données

Le prétraitement est réalisé dans le script `spark_ml/preprocessing.py`. Il transforme les données brutes en features exploitables par les modèles IA, tout en évitant toute fuite d'information entre train et test.

5.2.1 Encodage des variables catégorielles (LabelEncoder)

Les colonnes catégorielles `proto`, `service` et `state` sont encodées avec un `LabelEncoder` [9] fitté **uniquement** sur les données d'entraînement, puis appliqué au jeu de test. Cette approche évite toute fuite d'information (*data leakage*).

Listing 5.1 – Encodage des variables catégorielles

```
from sklearn.preprocessing import LabelEncoder

cat_cols = ['proto', 'service', 'state']
encoders = {}
for col in cat_cols:
    le = LabelEncoder()
```

```
    X_train[col] = le.fit_transform(X_train[col].astype(str))
    X_test[col]  = le.transform(X_test[col].astype(str))
    encoders[col] = le
```

5.2.2 Normalisation avec RobustScaler

La normalisation est effectuée avec un `RobustScaler` [9], choisi pour sa résistance aux valeurs aberrantes (*outliers*) fréquentes dans les données réseau. Il utilise la médiane et l'intervalle interquartile (IQR) plutôt que la moyenne et l'écart-type.

Listing 5.2 – Normalisation avec RobustScaler

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

5.2.3 Suppression des variables à variance nulle

Les colonnes dont la variance est nulle (valeur constante sur tout le dataset) sont supprimées car elles n'apportent aucune information discriminante au modèle.

Listing 5.3 – Suppression des features à variance nulle

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.0)
X_train = selector.fit_transform(X_train)
X_test  = selector.transform(X_test)
```

5.2.4 Traitement des valeurs extrêmes (clip ± 10)

Après normalisation, les valeurs aberrantes résiduelles sont écrêtées (*clipping*) dans l'intervalle $[-10,\ +10]$ pour limiter leur impact sur les modèles sensibles aux extrêmes (notamment le réseau de neurones).

Listing 5.4 – Écrêtage des valeurs extrêmes

```
import numpy as np

X_train = np.clip(X_train, -10, 10)
X_test  = np.clip(X_test,  -10, 10)
```

5.3 Modèle 1 - Random Forest (supervisé)

Le Random Forest [6] est un ensemble de 300 de décision entraînés par *bootstrap aggregating* (bagging). Chaque arbre est construit sur un sous-ensemble aléatoire des données et des features (`max_features="sqrt"`), ce qui réduit la corrélation entre arbres et améliore la généralisation.

Paramètres clés.

- `n_estimators = 300` : nombre d'arbres suffisant pour stabiliser la variance.
- `max_depth = 25`, `min_samples_leaf = 4`, `min_samples_split = 8` : contraintes anti-surapprentissage.
- `class_weight = "balanced"` : les poids des classes sont automatiquement inversement proportionnels à leur fréquence, ce qui compense le déséquilibre NORMAL/ATTACK.
- `oob_score = True` : le score *out-of-bag* fournit une estimation gratuite de la performance sur des données non vues durant l'entraînement.

Une validation croisée stratifiée en 5 plis est effectuée sur un sous-ensemble du jeu d'entraînement (30 000 exemples) pour vérifier la stabilité du modèle (variance inter-plis cible $< 0{,}03$).

5.4 Modèle 2 - Isolation Forest (non supervisé)

L'Isolation Forest [7]est un algorithme de détection d'anomalies semi-supervisé : il est entraîné **uniquement sur le trafic normal**, puis détecte les attaques comme des points aberrants difficiles à isoler.

Principe. L'algorithme construit des arbres de partitionnement aléatoire. Un point est considéré anormal si un nombre réduit de coupures suffit à l'isoler (chemin court dans l'arbre), tandis que les points normaux nécessitent davantage de coupures.

Paramètres clés.

- `n_estimators = 200`, `max_samples = 256` : valeurs standard optimales pour ce type de données.
- `contamination` : estimé dynamiquement à partir du ratio attaques/total dans le jeu d'entraînement (légèrement surestimé de 20 % pour capturer davantage d'anomalies).

Calibration du seuil. Pour éviter toute fuite de données, le seuil de décision est calibré sur un split de validation issu du jeu d'entraînement (15 %), en cherchant le seuil qui maximise le score F1. Ce même seuil est ensuite appliqué au jeu de test officiel.

Intérêt SIEM. Contrairement aux modèles supervisés, l'Isolation Forest peut détecter des attaques inconnues (*zero-day*) non présentes dans les données d'entraînement, ce qui en fait un complément indispensable en production.

5.5 Modèle 3 — Réseau de Neurones MLP (supervisé)

Le modèle neuronal utilise la classe `MLPClassifier` de scikit-learn avec une architecture à trois couches cachées.

Architecture. Le nombre de neurones par couche est adapté à la dimensionnalité des données ($n_features = 42$) :

$$\text{couches cachées} = (\min(256,\ 2 \times n_features),\ 128,\ 64)$$

soit une architecture large-intermédiaire-petit qui capte la complexité sans mémoriser les données.

Mécanismes anti-surapprentissage.

- `alpha = 0.01` : régularisation L2 forte pénalisant les poids trop élevés.
- `early_stopping = True` avec `validation_fraction = 0.12` et une patience de 20 epochs : l'entraînement s'arrête automatiquement si la perte sur le jeu de validation ne diminue plus.
- `learning_rate = "adaptive"` : le taux d'apprentissage diminue automatiquement en cas de stagnation.
- `batch_size = 512` : mini-batches qui introduisent du bruit stochastique bénéfique à la généralisation.

5.6 Modèle 4 - XGBoost (supervisé)

XGBoost [8](*eXtreme Gradient Boosting*) est un algorithme d'ensemble par boosting de gradient : les arbres sont construits séquentiellement, chacun corrigeant les erreurs du précédent.

Paramètres clés.

- `n_estimators = 3000` avec `early_stopping_rounds = 30` : le plafond est élevé mais l'arrêt précoce détermine le nombre optimal d'arbres sans surapprentissage.
- `max_depth = 6`, `min_child_weight = 5`, `gamma = 0.1` : contrôle de la complexité des arbres.
- `subsample = 0.8`, `colsample_bytree = 0.8` : sous-échantillonnage stochastique des lignes et des features (effet *bagging* intégré).
- `reg_alpha = 0.1`, `reg_lambda = 1.0` : régularisations L1 et L2.
- `scale_pos_weight` $= n_{\text{NORMAL}}/n_{\text{ATTACK}}$: compensateur du déséquilibre de classes.
- `eval_metric = "aucpr"` : la métrique d'arrêt précoce est l'aire sous la courbe Précision-Rappel, plus sensible au déséquilibre que l'AUC-ROC classique.
- `tree_method = "hist"` : algorithme histogramme rapide même sans GPU.

Un split de validation (15 %) est extrait du jeu d'entraînement pour l'arrêt précoce, garantissant l'absence de fuite vers le jeu de test.

5.7 Tableau comparatif

Précision, Rappel, F1-score

Le tableau 5.1 présente les performances des quatre modèles sur le jeu de test officiel UNSW-NB15 (82 332 flux, dont 37 000 normaux et 45 332 attaques).

TABLE 5.1 – Comparaison des modèles sur le test set UNSW-NB15

Modèle	Préc.	Rappel	F1	F1-macro	AUC-ROC	FP%	FN%
Random Forest	0,8555	0,9779	0,9126	0,8934	0,9848	20,24%	2,21%
Isolation Forest	0,6415	0,9804	0,7755	0,6307	0,7671	67,14%	1,96%
Neural Network	0,8111	0,9739	0,8850	0,8542	0,9736	27,79%	2,61%
XGBoost	**0,8718**	**0,9659**	**0,9165**	**0,9005**	**0,9836**	**17,40%**	**3,41%**

Les trois modèles supervisés (Random Forest, Neural Network, XGBoost) affichent des performances similaires et élevées. L'Isolation Forest, en tant que modèle non supervisé entraîné uniquement sur le trafic normal, présente un rappel excellent (98,04 %) mais un taux de faux positifs très élevé (67,14 %), ce qui est caractéristique des approches par détection d'anomalies.

5.7.1 Temps d'inférence

TABLE 5.2 – Temps d'inférence par échantillon sur le test set

Modèle	Temps (ms/échantillon)
Random Forest	0,0313
Isolation Forest	0,0113
Neural Network	0,0174
XGBoost	**0,0029**

XGBoost est le modèle le plus rapide à l'inférence (0,0029 ms/échantillon), ce qui est déterminant pour le traitement temps réel dans Spark Streaming.

5.8 Vote majoritaire et intégration dans le pipeline

5.8.1 Principe du vote majoritaire

Pour améliorer la robustesse et réduire les faux positifs isolés, les quatre modèles sont combinés via un **vote majoritaire à seuil configurable**. La décision finale pour chaque flux réseau est calculée comme suit :

$$\text{verdict}(x) = \begin{cases} \texttt{ATTACK} & \text{si } \sum_{m \in \mathcal{M}} \hat{y}_m(x) \geq \theta \\ \texttt{NORMAL} & \text{sinon} \end{cases}$$

où $\mathcal{M} = \{\text{RF, XGBoost, NN, IF}\}$ est l'ensemble des modèles, $\hat{y}_m(x) \in \{0, 1\}$ est la prédiction du modèle m pour le flux x, et $\theta = 2$ est le seuil de vote par défaut (majorité simple sur 4 modèles). En cas d'égalité exacte (2/4), la décision est `ATTACK` par prudence SIEM, afin de minimiser les attaques manquées.

Niveaux de sévérité. La sévérité d'une alerte est déterminée par le nombre de modèles ayant voté ATTACK :

TABLE 5.3 – Échelle de sévérité selon le vote

Votes ATTACK	Sévérité	Interprétation
4/4	`HIGH`	Unanimité — attaque très probable
3/4	`MEDIUM`	Fort consensus — alerte prioritaire
2/4	`LOW`	Seuil minimal atteint — à investiguer
0-1/4	`NORMAL`	Flux considéré normal

Score de confiance. Un indice de confiance est calculé comme la moyenne des probabilités de la classe ATTACK fournies par les trois modèles supervisés (RF, XGBoost, NN). L'Isolation Forest, produisant un score d'anomalie et non une probabilité calibrée, est exclu de ce calcul :

$$\text{confiance}(x) = \frac{1}{|\mathcal{M}_{\text{sup}}|} \sum_{m \in \mathcal{M}_{\text{sup}}} \hat{p}_m(x)$$

5.8.2 Architecture du pipeline Spark

Le fichier `elasticsearch_sender.py` implémente le pipeline complet de détection en temps réel. Son fonctionnement est le suivant :

1. **Lecture Kafka** : Spark Streaming consomme le topic `flux-reseau` (broker `localhost:9092`) en continu, par déclenchements toutes les 5 secondes et au maximum 1 000 messages par déclenchement.
2. **Parsing JSON** : chaque message est désérialisé selon un schéma fixe correspondant aux 44 colonnes du dataset UNSW-NB15.
3. **Prétraitement** (`foreachBatch`) : pour chaque micro-batch converti en DataFrame Pandas, le pipeline applique dans l'ordre : encodage des colonnes catégorielles (`proto`, `service`, `state`) via les `LabelEncoder` persistés, écrêtage des valeurs aberrantes ($\pm 10\sigma$), puis normalisation par le `RobustScaler` ajusté lors du prétraitement.
4. **Inférence parallèle** : les quatre modèles chargés en mémoire (RF, XGBoost, NN, IF) sont appliqués simultanément sur la matrice de features résultante.
5. **Vote majoritaire** : la fonction `majority_vote` agrège les prédictions individuelles et détermine le verdict final, la sévérité et le score de confiance pour chaque flux.
6. **Indexation Elasticsearch** : seuls les flux classés `ATTACK` sont indexés dans l'index `siem-alertes`. Chaque document contient le timestamp, le verdict, la sévérité, la confiance, les prédictions et scores individuels de chaque modèle, ainsi que les métadonnées réseau du flux (protocole, service, état, volume d'octets, durée).

5.8.3 Structure d'un document Elasticsearch

Un exemple de document indexé dans `siem-alertes` est présenté ci-dessous :


```
{
  "timestamp":  "2025-05-19T10:32:41.123456",
  "flow_id":    4821,
  "verdict":    "ATTACK",
  "votes":      3,
  "n_models":   4,
  "severite":   "MEDIUM",
  "confidence": 0.9134,
  "rf_pred":    1,  "rf_proba":  0.9520,
  "xgb_pred":   1,  "xgb_proba": 0.9741,
  "nn_pred":    1,  "nn_proba":  0.8141,
  "if_pred":    0,  "if_score":  0.3812,
  "proto":      "tcp",
  "service":    "http",
  "state":      "FIN",
```

```
  "attack_cat": "Exploits",
  "real_label": 1,
  "dur":        0.121,
  "sbytes":     4096,
  "dbytes":     512,
  "rate":       33.88
}
```

Le mapping Elasticsearch [5] est créé automatiquement à l'initialisation avec les types appropriés (`date`, `keyword`, `integer`, `float`) pour permettre des agrégations et visualisations dans Kibana.

5.9 Analyse et recommandation

Meilleur modèle global. XGBoost obtient le meilleur compromis avec un F1 de 0,9165, un AUC-ROC de 0,9836 et le temps d'inférence le plus faible (0,0029 ms/échantillon). Il est recommandé comme modèle principal de détection en production.

Complémentarité des modèles. L'Isolation Forest, malgré un taux de faux positifs élevé en évaluation isolée, joue un rôle complémentaire irremplaçable dans le vote : il est le seul modèle capable de détecter des attaques de type *zero-day* non présentes dans les données d'entraînement, car il ne s'appuie pas sur des exemples d'attaques étiquetés.

Architecture recommandée pour le SIEM.

- **Détection principale** : XGBoost, pour sa précision et sa rapidité.
- **Couverture zero-day** : Isolation Forest en parallèle.
- **Seuil d'alerte** : un rappel $\geq 0{,}90$ est recommandé pour un SIEM (mieux vaut déclencher trop d'alertes que manquer une attaque).
- **Vote majoritaire** : le seuil $\theta = 2/4$ offre un bon équilibre entre faux positifs et attaques manquées ; il peut être abaissé à $\theta = 1/4$ pour les environnements à très haute sensibilité sécuritaire.

Chapitre 6

Traitement avec Spark Streaming

Ce chapitre décrit l'implémentation du pipeline de traitement en temps réel basé sur Apache Spark Structured Streaming [4]. Ce composant constitue le cœur opérationnel du SIEM : il consomme les flux réseau depuis Kafka, applique les quatre modèles d'intelligence artificielle entraînés, agrège les résultats par fenêtres temporelles, et distribue les alertes vers Elasticsearch et MongoDB pour visualisation et archivage.

6.1 Test sur machine locale

6.1.1 Architecture Spark Structured Streaming

Le pipeline temps réel repose sur Apache Spark Structured Streaming [3], qui traite les données en continu sous forme de micro-batchs successifs. La session Spark est configurée en mode `local[*]`, ce qui permet d'exploiter l'ensemble des cœurs disponibles de la machine pour paralléliser le traitement et améliorer les performances. Au démarrage de l'application, le connecteur Spark–Kafka (`spark-sql-kafka`) est chargé dynamiquement afin de permettre la lecture directe des topics Kafka depuis Spark, sans couche intermédiaire.

La session Spark est ainsi initialisée avec un nom d'application spécifique au SIEM, une adresse de driver (par exemple `127.0.0.1`) et un nombre de partitions de shuffle adapté (par exemple quatre partitions) afin de contrôler la répartition des tâches lors des opérations de regroupement. Ces réglages assurent un compromis entre utilisation des ressources et stabilité du pipeline de traitement en continu.

Deux scripts Spark Structured Streaming sont ensuite exécutés en parallèle et lisent indépendamment le même topic Kafka, chacun avec son propre répertoire de checkpoint pour garantir la tolérance aux pannes. Le premier script, `elasticsearch_sender.py`, effectue la prédiction ligne par ligne, envoie les alertes vers l'index `siem-alertes` d'Elasticsearch et stocke tous les logs bruts dans MongoDB. Le second script, `windowing.py`, réalise une agrégation temporelle des flux par fenêtres et calcule des statistiques globales qui sont publiées dans un index Elasticsearch séparé, `siem-window-stats`, dédié au suivi des indicateurs de sécurité agrégés.

6.1.2 Lecture des flux depuis Kafka

Topic `flux-reseau`

Le stream est initialisé de manière à ne traiter que les nouveaux messages entrants produits après le démarrage de l'application. Pour cela, le paramètre `startingOffsets` est positionné sur `latest`, ce qui indique à Spark Structured Streaming de commencer

la lecture à partir des derniers offsets du topic Kafka plutôt que de relire l'historique complet. Cette configuration correspond à un scénario de supervision en temps réel, où seules les données récentes sont pertinentes pour la détection d'incidents de sécurité.

Afin de contrôler le débit de traitement et d'éviter toute surcharge mémoire au niveau des micro-batchs, un plafond est défini sur le nombre de messages pris en compte à chaque déclenchement du traitement. Le paramètre `maxOffsetsPerTrigger` est ainsi fixé à une valeur de 1000, ce qui limite le volume de données consommé par micro-batch et permet de stabiliser la latence tout en préservant les ressources de la plateforme de streaming.

Parsing JSON et schéma des données

Les messages produits dans Kafka arrivent dans Spark sous forme d'octets bruts, ce qui nécessite une étape de conversion avant de pouvoir les exploiter. La valeur transportée par chaque message est d'abord transformée en chaîne de caractères (`STRING`), puis désérialisée au format JSON à l'aide de la fonction `from_json()`. Cette désérialisation s'appuie sur un schéma Spark strictement défini à l'avance, ce qui garantit une interprétation cohérente de tous les champs du flux.

Le schéma retenu comporte 45 champs qui correspondent exactement aux colonnes du jeu de données UNSW-NB15 utilisé lors de la phase d'entraînement. Pour chacun de ces champs, le type de données est spécifié de manière explicite (par exemple `LongType` pour les entiers, `DoubleType` pour les valeurs réelles et `StringType` pour les chaînes de caractères). Cette définition précise des types permet d'optimiser l'utilisation de la mémoire et d'éviter les inférences automatiques de schéma, qui seraient coûteuses en temps de calcul et risqueraient d'introduire des incohérences.

Après l'application de `from_json()`, le contenu JSON est projeté dans un DataFrame structuré où chaque champ du schéma devient une colonne distincte. On obtient ainsi directement une vue tabulaire des flux réseau, avec des colonnes telles que l'identifiant de flux, la durée, le protocole, le service, l'état, les compteurs de paquets et d'octets, ainsi que les colonnes `attack_cat` et `label` qui décrivent la nature de l'événement dans le dataset d'origine.

6.1.3 Prétraitement en streaming

Chargement des artefacts de prétraitement

Avant le démarrage du pipeline de streaming, l'ensemble des artefacts générés par le script `preprocessing.py` est chargé une seule fois en mémoire lors de la phase d'initialisation. Il s'agit notamment du scaler utilisé pour la normalisation des variables, de la liste ordonnée des caractéristiques (*feature names*), des encodeurs dédiés aux variables catégorielles ainsi que des paramètres de découpe des valeurs extrêmes (clipping).

Ce chargement "à froid" présente un avantage important : il évite de sérialiser et désérialiser les objets `scikit-learn` à chaque micro-batch traité. Si ces artefacts étaient relus depuis le disque à chaque itération, le coût d'accès et de décompression serait prohibitif en termes de latence et pénaliserait fortement le fonctionnement temps réel du SIEM. En centralisant leur chargement au démarrage, le pipeline peut ensuite réutiliser directement ces objets en mémoire pour traiter rapidement chaque nouveau lot de données.

Transformation des micro-batchs

À chaque micro-batch, les données du flux sont regroupées dans un DataFrame puis passées à une fonction de prétraitement qui reproduit exactement les transformations

utilisées lors de l'entraînement.

Cette fonction applique successivement :

— la suppression des colonnes non utilisées pour la prédiction (`id`, `attack_cat`, `label`) ;
— l'encodage des colonnes catégorielles (`proto`, `service`, `state`) avec des *LabelEncoders* pré-entraînés, en remplaçant les valeurs inconnues par la modalité la plus fréquente ;
— la conversion de toutes les colonnes en valeurs numériques, avec remplacement des valeurs manquantes par zéro ;
— un « clipping » des valeurs extrêmes autour de la moyenne, dans l'intervalle $[\mu - 10\sigma,\ \mu + 10\sigma]$ pour chaque caractéristique ;
— la normalisation finale des variables à l'aide du `RobustScaler`, afin d'obtenir une matrice de caractéristiques prête pour les modèles de détection.

6.1.4 Chargement des modèles sauvegardés

Les quatre modèles de détection sont chargés au démarrage du pipeline à partir des fichiers sauvegardés dans le répertoire `artifacts/models`. Chaque fichier contient un modèle entraîné hors ligne et sérialisé au format `pkl`. L'`IsolationForest` est stocké sous la forme d'une structure contenant à la fois l'objet du modèle et un seuil d'anomalie (calibré séparément lors de la phase d'entraînement), généralement représentée par un dictionnaire de type {`"model": ..., "threshold": ...`}. Ce seuil permet de convertir le score d'anomalie continu en décision binaire (normal ou anormal) au moment de l'inférence.

Afin d'uniformiser l'interface d'inférence, les modèles supervisés (Random Forest, XGBoost et réseau de neurones) sont encapsulés dans le même format que l'`IsolationForest`. Pour ces modèles, le champ `model` contient l'algorithme entraîné et le champ `threshold` est renseigné à `None`, car la décision repose directement sur la probabilité de classe. On obtient ainsi une structure de données unique qui associe, pour chaque nom de modèle, un couple `(model, threshold)`. Cette homogénéisation simplifie les boucles de chargement et de prédiction, et facilite la mise en œuvre du vote majoritaire entre les différents modèles.

Modèle	Type	Stockage
Random Forest	Supervisé	{"model": objet RF, "threshold": None}
XGBoost	Supervisé	{"model": objet XGB, "threshold": None}
Réseau de neurones	Supervisé	{"model": objet NN, "threshold": None}
Isolation Forest	Non supervisé	{"model": objet IF, "threshold": seuil d'anomalie}

TABLE 6.1 – Structure de stockage des modèles de détection

6.1.5 Inférence sur les micro-batchs

Pour chaque micro-batch prétraité, une fonction d'inférence parcourt séquentiellement les quatre modèles chargés. Les modèles supervisés (Random Forest, XGBoost et réseau de neurones) produisent, pour chaque flux réseau, d'une part une classe prédite (ATTACK

ou NORMAL) via la méthode de prédiction standard, et d'autre part un score de confiance associé à la classe d'attaque, obtenu par la probabilité de sortie. Ces informations sont conservées pour chaque modèle afin de pouvoir être utilisées lors de la phase de vote ultérieure.

L'`IsolationForest` suit une logique différente, car il fournit d'abord un score d'anomalie continu plutôt qu'une probabilité directe. Ce score, de signe négatif, est inversé puis comparé au seuil d'anomalie calibré lors de l'entraînement pour produire une prédiction binaire (normal ou anormal). Afin de rendre les résultats comparables à ceux des modèles supervisés, les scores d'anomalie sont ensuite normalisés dans l'intervalle $[0, 1]$, ce qui permet de les interpréter comme un score de risque relatif pour chaque flux.

Modèle	Sortie principale	Score associé
Random Forest	Classe prédite (0/1)	Probabilité de la classe ATTACK (`predict_proba`)
XGBoost	Classe prédite (0/1)	Probabilité de la classe ATTACK (`predict_proba`)
Réseau de neurones	Classe prédite (0/1)	Probabilité de la classe ATTACK en sortie du modèle
Isolation Forest	Décision binaire (normal/anormal)	Score d'anomalie normalisé dans $[0, 1]$

TABLE 6.2 – Type de sortie et score de confiance pour chaque modèle

6.1.6 Vote majoritaire

La décision finale repose sur un vote majoritaire entre les quatre modèles de détection. Un seuil de vote, noté `N_VOTE_THRESHOLD`, est fixé à 2 : un flux est classé `ATTACK` si au moins deux modèles sur quatre votent positivement. Ce choix permet de trouver un compromis entre sensibilité et spécificité : un seuil de 1 rendrait le système trop sensible et générerait de nombreux faux positifs, tandis qu'un seuil de 3 ou 4 rendrait la détection trop stricte et risquerait de laisser passer des attaques réelles. Le vote majoritaire fournit ainsi une décision globale plus robuste que celle de n'importe quel modèle pris individuellement.

Le score de confiance associé à cette décision est calculé à partir des seuls modèles supervisés (Random Forest, XGBoost et réseau de neurones). Pour chaque flux, les probabilités d'appartenance à la classe `ATTACK` retournées par ces modèles sont moyennées afin d'obtenir un indicateur unique de confiance. L'`IsolationForest` n'est pas inclus dans ce calcul, car il produit des scores d'anomalie qui ne sont pas des probabilités calibrées. Le système dispose ainsi, pour chaque flux, à la fois d'un verdict binaire (`ATTACK` ou `NORMAL`) et d'un score de confiance compris entre 0 et 1.

Élément	Rôle
Seuil de vote (= 2)	Flux classé `ATTACK` si au moins 2 modèles sur 4 votent positivement.
Seuil = 1	Très sensible, nombreux faux positifs.
Seuil = 3 ou 4	Très strict, risque de manquer des attaques.
Score de confiance	Moyenne des probabilités des modèles supervisés uniquement.
Isolation Forest	Utilisé pour le vote, mais exclu du calcul de confiance (score d'anomalie non calibré).

TABLE 6.3 – Principe du vote majoritaire et du calcul du score de confiance

6.1.7 Attribution du niveau de sévérité

Le niveau de sévérité est attribué proportionnellement au nombre de modèles ayant voté `ATTACK`. Un consensus unanime, c'est-à-dire quatre votes sur quatre, déclenche une alerte `HIGH`. Lorsque trois modèles sur quatre votent positivement, la sévérité est fixée à `MEDIUM`. Enfin, le seuil minimal de deux votes donne une sévérité `LOW`, tandis qu'un nombre de votes inférieur à ce seuil correspond à un état `NORMAL`. Cette graduation permet de hiérarchiser les alertes selon leur niveau de consensus et donne une lecture plus fine du risque détecté.

En parallèle, la console affiche en temps réel un résumé de chaque micro-batch traité afin de faciliter le suivi opérationnel du pipeline. Pour chaque modèle, le système indique le nombre d'attaques détectées ainsi que la probabilité moyenne associée aux prédictions. Un récapitulatif global est également présenté pour la répartition des niveaux de sévérité (`HIGH`, `MEDIUM` et `LOW`), ce qui permet de surveiller rapidement la charge d'alertes et l'intensité des attaques détectées sur le flux en cours.

Nombre de votes `ATTACK`	Sévérité attribuée
4/4	`HIGH`
3/4	`MEDIUM`
2/4	`LOW`
0/4 ou 1/4	`NORMAL`

Table 6.4 – Attribution du niveau de sévérité selon le nombre de votes

Information affichée	Utilité
Nombre d'attaques par modèle	Suivi détaillé des décisions individuelles.
Probabilité moyenne par modèle	Estimation rapide de la confiance du modèle.
Répartition des sévérités	Vue d'ensemble de l'intensité des alertes.
Résumé par batch	Contrôle en temps réel du comportement du pipeline.

Table 6.5 – Résumé console des informations affichées pour chaque batch

6.1.8 Stockage des résultats

Sauvegarde des logs bruts dans MongoDB

Tous les flux traités, qu'ils soient classés `ATTACK` ou `NORMAL`, sont insérés dans la collection `logs_bruts` de MongoDB. Chaque document contient les principales caractéristiques du flux réseau ainsi que les résultats complets du pipeline de détection, notamment le vote final, le nombre total de votes, le verdict, le niveau de sévérité et le score de confiance. Cette collection joue ainsi le rôle de mémoire à long terme du SIEM, en conservant l'ensemble des événements pour d'éventuelles analyses forensiques ou investigations ultérieures.

En parallèle, les flux détectés comme `ATTACK` sont également dupliqués dans la collection `alertes`. Cette seconde collection enrichit les données stockées avec les prédictions individuelles de chaque modèle (`rf_pred`, `xgb_pred`, `nn_pred` et `if_pred`), afin de faciliter la traçabilité et l'audit des décisions prises par le système. Elle permet de savoir précisément quels modèles ont contribué à une alerte donnée et de reconstituer le raisonnement du pipeline en cas d'analyse approfondie.

Champ stocké	Rôle
`flow_id`	Identifiant unique du flux.
`proto`, `service`, `state`	Caractéristiques réseau principales.
`sbytes`, `dbytes`, `rate`	Indicateurs de volume et de débit.
`vote_final`, `votes`	Résultat du vote majoritaire.
`verdict`, `severite`	Décision finale et niveau de criticité.
`confidence`	Score global associé à la prédiction.
`timestamp`	Date et heure de traitement.

TABLE 6.6 – Principaux champs stockés dans la collection `logs_bruts`

Envoi des alertes vers Elasticsearch

Seuls les flux classés `ATTACK` par le vote majoritaire sont indexés dans Elasticsearch, au sein de l'index `siem-alertes`. Lors du premier lancement, le mapping de cet index est créé de manière programmatique avec des types de champs explicitement définis. Les champs destinés au filtrage et à la navigation dans Kibana, tels que le verdict, la sévérité ou le protocole, sont déclarés comme `keyword`, tandis que l'horodatage est stocké au format `date`. Les métriques numériques, comme les probabilités, les scores et les prédictions individuelles des modèles, sont quant à elles typées en `float` ou `integer` selon leur nature.

Cette structuration explicite du mapping améliore les performances d'indexation et facilite les requêtes analytiques dans Kibana, notamment les filtres, les agrégations et les recherches temporelles. L'envoi des alertes vers Elasticsearch est effectué en mode *bulk*, ce qui permet de regrouper plusieurs documents dans une seule opération d'écriture et de réduire le coût lié aux insertions répétées. Cette approche est particulièrement adaptée à un pipeline de détection en continu, car elle combine rapidité d'indexation et bonne capacité de traitement des flux d'alertes.

Champ	Type Elasticsearch	Rôle
`timestamp`	`date`	Analyse temporelle des alertes.
`verdict`, `severite`, `proto`, `service`, `attack_cat`	`keyword`	Filtrage et regroupement dans Kibana.
`confidence`, `rf_proba`, `xgb_proba`, `nn_proba`, `if_score`	`float`	Scores et probabilités.
`rf_pred`, `xgb_pred`, `nn_pred`, `if_pred`	`integer`	Prédictions individuelles des modèles.

TABLE 6.7 – Structure du mapping Elasticsearch pour les alertes SIEM

6.1.9 Calcul des statistiques par fenêtres temporelles

Fenêtres glissantes (1 minute / 30 secondes)

Le script `windowing.py` exécute un second pipeline Spark Streaming en parallèle du script principal de détection. Le pipeline A utilise un *trigger* toutes les 30 secondes, ce qui permet de construire une fenêtre glissante d'une minute avec un pas de 30 secondes. Pour chaque fenêtre, les flux reçus sont analysés puis agrégés afin de produire des statistiques globales sur l'activité réseau.

Un score de menace, compris entre 0 et 100, est calculé pour résumer le niveau de risque observé dans la fenêtre. Ce score combine le taux d'attaque et la répartition des niveaux de sévérité, en accordant plus de poids aux alertes les plus critiques. Une alerte critique est levée automatiquement si le taux d'attaque dépasse 30 % ou si au moins une attaque de niveau `HIGH` est détectée dans la fenêtre.

Indicateur	Rôle
`window_start`, `window_end`	Délimitation temporelle de la fenêtre.
`total_flux`, `total_attack`	Nombre total de flux et nombre de flux attaquants.
`taux_attaque_pct`	Pourcentage de flux classés `ATTACK`.
`threat_score`	Score global de menace de 0 à 100.
`high_count`, `medium_count`, `low_count`	Répartition des niveaux de sévérité.
`top_proto`, `top_attack_cat`	Protocole et catégorie d'attaque les plus fréquents.
`total_sbytes`, `total_dbytes`	Volume total de trafic échangé.

Table 6.8 – Principaux indicateurs calculés pour chaque fenêtre glissante

Fenêtres tumbling (5 minutes)

Le pipeline B utilise un *trigger* de 5 minutes afin de produire des snapshots temporels non chevauchants. Contrairement à la fenêtre glissante, le début de fenêtre est arrondi à la tranche de 5 minutes la plus proche, ce qui facilite l'alignement des résultats sur une chronologie régulière. Cette organisation est particulièrement adaptée à la visualisation dans Kibana, où les tableaux de bord gagnent en lisibilité lorsque les points de mesure sont positionnés sur des intervalles fixes.

Les deux pipelines s'exécutent simultanément sur le même flux Kafka, tout en conservant des checkpoints indépendants. Cette séparation permet à chaque traitement de progresser et de se récupérer sans interférer avec l'autre, ce qui améliore la robustesse globale du système. Le pipeline glissant fournit une vision fine et réactive de l'activité, tandis que le pipeline tumbling offre une lecture plus synthétique et régulière de l'évolution du trafic.

Élément	Rôle
`processingTime = 5 minutes`	Déclenchement périodique du pipeline tumbling.
`window_start` arrondi	Alignement des snapshots sur des tranches régulières de 5 minutes.
Snapshot non chevauchant	Vue synthétique de l'activité sur une période fixe.
Checkpoints indépendants	Reprise autonome de chaque pipeline en cas d'arrêt.

Table 6.9 – Caractéristiques du pipeline tumbling

6.1.10 Exécution continue du pipeline

L'ensemble du pipeline s'exécute en continu jusqu'à interruption manuelle, grâce à un mécanisme d'attente de terminaison qui maintient les traitements en cours tant que l'application n'est pas arrêtée. Pour le script de détection principal, le déclenchement est fixé à 5 secondes afin de réduire la latence entre l'arrivée des données et la génération des alertes, ce qui permet une réponse quasi temps réel. En parallèle, des répertoires de checkpoint distincts sont utilisés pour les différents pipelines afin de garantir la reprise automatique du traitement en cas d'arrêt, sans perte des données déjà consommées.

L'arrêt du système est géré proprement lorsque l'utilisateur interrompt l'exécution, notamment via `Ctrl+C`. Dans ce cas, le flux en cours est arrêté, la session Spark est fermée, puis la connexion à MongoDB est correctement libérée. Cette gestion soignée de l'arrêt évite les blocages et assure la cohérence des ressources utilisées par le SIEM pendant toute la durée d'exécution.

Élément	Rôle
`awaitTermination()`	Maintien du pipeline en exécution continue.
`processingTime = 5 seconds`	Réduction de la latence de détection.
`checkpointLocation`	Reprise du traitement après redémarrage.
Arrêt via `Ctrl+C`	Interruption manuelle propre du système.
Fermeture de Spark et MongoDB	Libération correcte des ressources.

Table 6.10 – Gestion de l'exécution continue et de l'arrêt propre du pipeline

latex

6.2 Test en architecture distribuée (2 machines)

Après validation du pipeline sur machine locale, nous avons procédé au déploiement en architecture distribuée sur deux machines physiques interconnectées en réseau local (LAN). Cette configuration reflète un environnement de production réaliste et permet d'évaluer les performances du système dans des conditions de parallélisme réel.

6.2.1 Choix de l'architecture et comparaison

Le passage à une architecture distribuée a nécessité une analyse comparative préalable entre le déploiement mono-machine et le cluster distribué. La figure 6.1 illustre les différences fondamentales entre ces deux approches selon plusieurs critères clés.

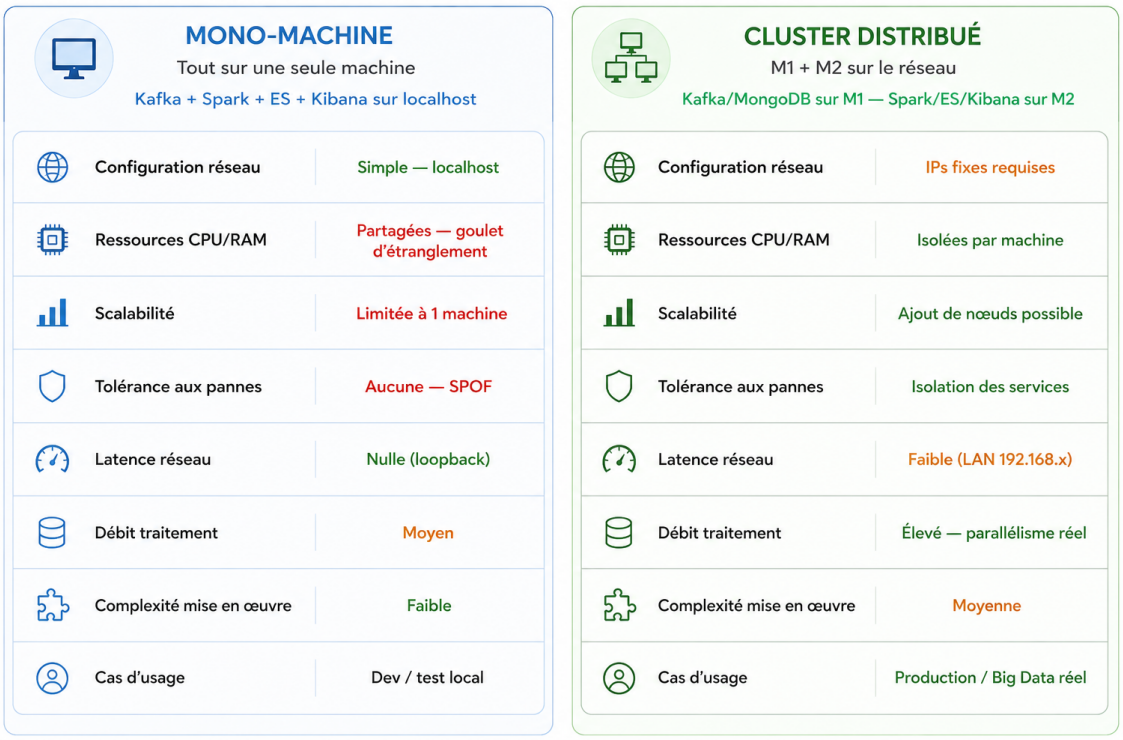


FIGURE 6.1 – Comparaison entre architecture mono-machine et cluster distribué

L'architecture mono-machine, bien qu'adaptée au développement local, présente des limitations rédhibitoires pour la production : ressources CPU/RAM partagées créant des goulets d'étranglement, scalabilité limitée à un seul hôte, et absence de tolérance aux pannes (SPOF). En revanche, le cluster distribué offre une isolation des ressources par machine, un parallélisme réel garantissant un débit de traitement élevé, et une tolérance aux pannes par isolation des services. C'est pourquoi l'architecture distribuée a été retenue pour cette phase de test.

6.2.2 Schéma du déploiement distribué

Le déploiement s'appuie sur deux machines physiques avec une répartition claire des responsabilités, comme illustré dans la figure 6.2.

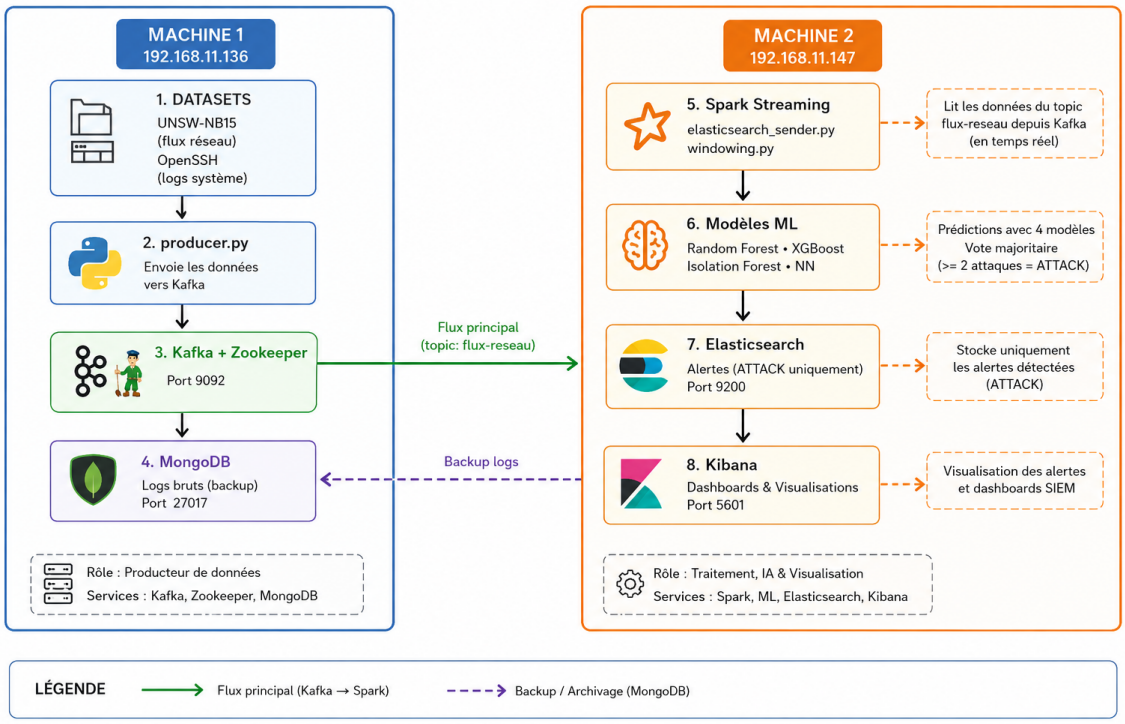


FIGURE 6.2 – Schéma du pipeline distribué : Machine 1 (producteur) et Machine 2 (traitement IA & visualisation)

La **Machine 1** (`192.168.11.136`) joue le rôle de producteur de données. Elle héberge les datasets UNSW-NB15 et OpenSSH, le script `producer.py` chargé d'envoyer les événements vers Kafka, le broker **Kafka + Zookeeper** (port 9092), ainsi que **MongoDB** (port 27017) pour l'archivage des logs bruts.

La **Machine 2** (`192.168.11.147`) est dédiée au traitement, à l'intelligence artificielle et à la visualisation. Elle exécute **Spark Structured Streaming** qui consomme le topic `flux-reseau` en temps réel, les **modèles ML** (Random Forest, XGBoost, Isolation Forest, réseau de neurones) avec vote majoritaire, **Elasticsearch** (port 9200) pour le stockage des alertes ATTACK, et **Kibana** (port 5601) pour la visualisation des dashboards SIEM.

Le flux principal circule de Kafka (Machine 1) vers Spark (Machine 2) via le topic `flux-reseau`. Un flux secondaire assure le backup des logs bruts vers MongoDB par un transfert en pointillés depuis Machine 2 vers Machine 1.

6.2.3 Configuration réseau et déploiement

La communication inter-machines repose sur le réseau local `192.168.11.x`. Sur la Machine 2, le consommateur Spark est configuré pour pointer vers l'adresse IP fixe de la Machine 1 afin d'accéder au broker Kafka :

Chaque machine dispose de son propre fichier `docker-compose.yml`. Sur la Machine 1, Docker Compose orchestre Kafka, Zookeeper et MongoDB avec un réseau interne `kafka-net`. Sur la Machine 2, il gère Spark (master + worker), Elasticsearch et Kibana, avec les paramètres mémoire ajustés (`ES_JAVA_OPTS=-Xms512m -Xmx512m`).

Chapitre 7

Stockage NoSQL

Ce projet repose sur deux moteurs de stockage NoSQL complémentaires : **MongoDB**, chargé de la persistance brute et opérationnelle, et **Elasticsearch**, orienté vers la recherche et l'indexation des alertes. Leurs rôles sont distincts et se complètent tout au long du pipeline de traitement.

7.1 MongoDB

MongoDB est utilisé comme base de persistance principale pour les données non structurées et semi-structurées issues du pipeline Kafka. Il stocke l'intégralité des logs réseaux entrants, les alertes de sécurité en copie de sauvegarde, ainsi que les statistiques agrégées de session. La base de données est nommée `siem_db` et s'exécute sur `localhost:27017`. Elle expose trois collections principales décrites ci-dessous.

7.1.1 Collection `logs_bruts`

La collection `logs_bruts` stocke chaque événement réseau reçu depuis Kafka sous la forme d'un document JSON complet. Chaque document conserve l'ensemble des 45 champs du dataset UNSW-NB15 (protocole, durée, octets source/destination, état de la connexion, etc.) auxquels s'ajoutent les métadonnées d'ingestion :

— `kafka_topic` : nom du topic Kafka source (`flux-reseau`) ;
— `timestamp` : horodatage de l'événement réseau ;
— `ingere_le` : horodatage UTC d'insertion dans MongoDB.

La fonction `sauvegarder_log_brut()` assure l'insertion unitaire, tandis que `sauvegarder_batch_log` réalise des insertions en lot (*bulk insert*) via `insert_many()`, réduisant la latence d'écriture lors de pics de trafic.

Un exemple de document inséré est présenté ci-dessous :

```
{
  "id": 999,
  "dur": 0.5,
  "proto": "tcp",
  "service": "http",
  "state": "FIN",
  "spkts": 10,
  "dpkts": 8,
  "sbytes": 1500,
  "dbytes": 300,
```

```
  "label": 1,
  "attack_cat": "Exploits",
  "timestamp": "2026-05-19T00:00:00",
  "kafka_topic": "flux-reseau",
  "ingere_le": "2026-05-19T00:00:01"
}
```

7.1.2 Collection `alertes`

La collection `alertes` constitue un miroir de sauvegarde des alertes de sécurité générées par le pipeline Spark et indexées dans Elasticsearch. Elle garantit la résilience du système : en cas d'indisponibilité temporaire d'Elasticsearch, les alertes restent accessibles via MongoDB.

Chaque document de cette collection reprend les champs de prédiction issus des modèles de machine learning (Random Forest, Isolation Forest, réseau de neurones), le vote majoritaire final (`vote_final`), le niveau de sévérité (`severite` : `LOW`, `MEDIUM`, `HIGH` ou `NORMAL`), ainsi qu'un champ `sauvegarde_le` indiquant l'horodatage de l'archivage.

7.1.3 Collection `stats_sessions`

La collection `stats_sessions` est réservée aux statistiques agrégées calculées lors du traitement par fenêtres temporelles (*windowing*) dans Spark. Elle peut stocker des indicateurs synthétiques tels que le nombre total de connexions par protocole, le ratio d'attaques par intervalle de temps, ou encore les volumes d'octets échangés, offrant ainsi une vue consolidée à des fins de supervision et de reporting.

7.1.4 Index et optimisation

Trois index MongoDB sont créés automatiquement à l'initialisation de la base, via la fonction `_creer_index()` :

TABLE 7.1 – Index MongoDB créés sur la collection `logs_bruts`

Nom de l'index	Champ(s)	Objectif
`idx_timestamp`	`timestamp` (ASC)	Tri et requêtes chronologiques
`idx_label`	`label` (ASC)	Filtrage rapide attaques / normaux
`idx_proto_service`	`proto` + `service` (ASC)	Agrégations réseau

Ces index réduisent significativement les temps de réponse des requêtes de monitoring (filtrage par label, tri temporel, agrégation par protocole) au fur et à mesure que le volume de données augmente.

7.2 Elasticsearch

Elasticsearch joue le rôle de moteur d'indexation et de recherche pour les alertes de sécurité. Ses capacités de recherche plein texte, de filtrage sur des champs typés et d'agrégation en temps réel en font un composant central pour la visualisation dans Kibana et l'interrogation rapide du SIEM.

7.2.1 Index `siem-alertes`

L'index Elasticsearch est nommé `alertes-securite` dans l'implémentation du projet. Il est configuré avec un seul shard (`number_of_shards: 1`) et aucun réplica (`number_of_replicas: 0`), ce qui convient à un déploiement mono-nœud en environnement de développement. La création de l'index est idempotente : si l'index existe déjà, aucune action n'est entreprise.

7.2.2 Mapping des champs

Le mapping définit explicitement le type de chaque champ pour garantir une indexation cohérente et des agrégations précises. Le tableau 7.2 présente les champs principaux et leur type Elasticsearch :

TABLE 7.2 – Mapping des champs de l'index `alertes-securite`

Champ	Type ES	Description
`timestamp`	`date`	Horodatage de l'événement
`ip_src`	`ip`	Adresse IP source
`ip_dst`	`ip`	Adresse IP destination
`port_src`	`integer`	Port source
`port_dst`	`integer`	Port destination
`proto`	`keyword`	Protocole réseau (tcp, udp, ...)
`service`	`keyword`	Service applicatif (http, ftp, ...)
`state`	`keyword`	État de la connexion (FIN, SYN, ...)
`label`	`integer`	Étiquette réelle (0 = normal, 1 = attaque)
`attack_cat`	`keyword`	Catégorie d'attaque
`prediction_rf`	`integer`	Prédiction Random Forest
`prediction_if`	`integer`	Prédiction Isolation Forest
`prediction_nn`	`integer`	Prédiction réseau de neurones
`vote_final`	`integer`	Vote majoritaire des modèles
`score_rf`	`float`	Score de probabilité (RF)
`score_nn`	`float`	Score de probabilité (NN)
`duree`	`float`	Durée de la connexion
`sbytes`	`long`	Octets envoyés par la source
`dbytes`	`long`	Octets envoyés par la destination
`severite`	`keyword`	Sévérité (LOW / MEDIUM / HIGH / NORMAL)
`source`	`keyword`	Topic Kafka d'origine

Le champ `severite` est calculé dynamiquement lors de l'envoi de chaque alerte, selon les règles suivantes :

- `NORMAL` : si le vote final est 0 (aucune attaque détectée) ;
- `HIGH` : si le score de confiance est supérieur ou égal à 0,85 ;
- `MEDIUM` : si le score est compris entre 0,60 et 0,85 ;
- `LOW` : si le score est inférieur à 0,60 mais que l'attaque est confirmée par vote majoritaire.

7.2.3 Insertion en bulk

Pour absorber les volumes d'alertes générés en continu par le pipeline Spark, la fonction `envoyer_batch()` utilise l'API `helpers.bulk()` de la bibliothèque Python `elasticsearch`. Cette approche réduit le nombre d'appels HTTP vers le cluster Elasticsearch et améliore significativement le débit d'indexation par rapport à des insertions unitaires successives.

Chaque action bulk précise l'index cible (`_index`) et le document source (`_source`), ce dernier incluant le calcul automatique du vote final. Les erreurs d'indexation sont capturées individuellement sans interrompre le traitement du lot, grâce au paramètre `raise_on_error=False`.

7.3 Contrôle de qualité des données

Avant tout entraînement des modèles, une analyse exhaustive de la qualité du dataset UNSW-NB15 a été réalisée à l'aide du script `data_quality.py`. Quatre dimensions ont été examinées.

7.3.1 Valeurs manquantes

L'analyse révèle que le dataset ne contient **aucune valeur manquante** : 0 valeur nulle a été détectée sur l'ensemble des 175 341 entrées d'entraînement et 82 332 entrées de test. Le dataset UNSW-NB15 est donc directement exploitable sans imputation préalable, ce qui simplifie significativement la phase de prétraitement.

7.3.2 Doublons

Le contrôle des doublons confirme l'absence totale d'entrées dupliquées dans les deux partitions (entraînement et test). Aucun traitement de déduplication n'a donc été nécessaire, garantissant l'intégrité statistique des ensembles d'évaluation.

7.3.3 Déséquilibre des classes

La distribution des classes révèle un déséquilibre notable entre les deux catégories du champ `label` :

— **Trafic normal (label = 0)** : 56 000 entrées, soit 31,9 % du jeu d'entraînement ;
— **Trafic d'attaque (label = 1)** : 119 341 entrées, soit 68,1 % du jeu d'entraînement ;
— **Ratio normal/attaque** : 0,47 :1.

Ce déséquilibre, bien qu'inverse à la situation classique (où les attaques sont rares), a été pris en compte lors de l'entraînement via le paramètre `class_weight` des modèles scikit-learn et l'utilisation de métriques adaptées telles que le F1-score macro et le recall par classe.

7.3.4 Valeurs infinies et variance nulle

L'inspection des valeurs numériques ne révèle **aucune valeur infinie** dans le dataset, évitant ainsi les problèmes de convergence qui peuvent survenir lors de l'entraînement de modèles basés sur des gradients.

De même, aucune colonne à variance nulle (features constantes inutiles) n'a été identifiée parmi les variables numériques, ce qui confirme que l'ensemble des 45 features du dataset apporte une information discriminante potentiellement exploitable par les classifieurs.

Les colonnes catégorielles (`proto`, `service`, `state`) ont été encodées avec un `LabelEncoder` ajusté uniquement sur les données d'entraînement, afin d'éviter toute fuite d'information (*data leakage*) vers l'ensemble de test. La normalisation des features numériques a été réalisée avec un `RobustScaler`, dont la robustesse aux valeurs aberrantes — très

fréquentes dans les données réseau (rafales, connexions anormalement longues) — en fait le choix le plus adapté au contexte SIEM.

Chapitre 8

Visualisation et Dashboards Kibana

8.1 Configuration des index patterns

Avant de créer les tableaux de bord, deux index patterns ont été configurés dans Kibana afin de rendre les données Elasticsearch interrogeables depuis l'interface de visualisation :

- `alertes-securite*` : indexe les alertes de sécurité générées par le vote majoritaire des modèles IA, avec le champ `timestamp` comme référence temporelle principale.
- `siem-window-stats*` : indexe les statistiques agrégées produites par les fenêtres temporelles du script `windowing.py`, permettant le suivi des indicateurs de menace par intervalles de temps.

La création de ces index patterns permet à Kibana d'inférer automatiquement les types de champs (dates, mots-clés, valeurs numériques) et de les rendre disponibles pour les visualisations et les filtres dynamiques.

8.2 Dashboard des alertes de sécurité

Le premier tableau de bord, dédié aux alertes de sécurité en temps réel, regroupe plusieurs visualisations complémentaires :

- **Répartition des protocoles réseau** : un graphique en anneau (*donut chart*) affiche la distribution des protocoles réseau détectés dans les alertes. Les protocoles UDP (42,96 %) et TCP (41,35 %) dominent le trafic, suivis par UNAS (6,28%), OSPF (1,14%) et SCTP (0,58%), ce qui reflète fidèlement la composition du dataset UNSW-NB15.
- **Distribution des types d'attaques** : un graphique circulaire présente la répartition des catégories d'attaques détectées. Les attaques de type *Generic* représentent la part la plus importante (33,69 %), suivies des flux normaux (20,02 %), des *Exploits* (19,84 %), des *Fuzzers* (9,92%) et des attaques *DoS* (7,3%).
- **Nombre de flux par protocole** : un graphique en barres horizontales compare le volume d'enregistrements, la moyenne des octets source/destination, la durée moyenne et le taux moyen pour chaque protocole. Cette visualisation permet d'identifier rapidement les protocoles les plus actifs et les plus susceptibles de véhiculer des attaques.
- **Attaques par protocole et catégorie** : un graphique en barres empilées croise le protocole réseau avec la catégorie d'attaque associée. Cette vue met en évidence que les protocoles UDP et TCP concentrent la majorité des attaques détectées, toutes catégories confondues.

— **Score de confiance médian** : un indicateur de type jauge affiche la médiane du score de confiance global des prédictions, permettant d'évaluer en un coup d'œil le niveau de certitude moyen des alertes générées.
— **Dernier score d'anomalie (if_score)** : une jauge horizontale affiche la dernière valeur du score d'anomalie produit par l'Isolation Forest, offrant une lecture instantanée du niveau d'anomalie sur le flux le plus récent.

8.3 Dashboard des statistiques temporelles

Le second tableau de bord est consacré à l'analyse temporelle du trafic réseau et des alertes :

— **Évolution temporelle des flux par type d'attaque** : un graphique en courbes représente le nombre d'enregistrements par fenêtre de 5 minutes, décomposé par catégorie d'attaque (Generic, Normal, Exploits, Other). Cette visualisation permet de détecter les pics d'activité malveillante et d'observer leur évolution dans le temps.
— **Volume de votes et nombre de flux dans le temps** : un graphique en barres groupées affiche simultanément la somme des votes ATTACK et le nombre total d'enregistrements par intervalle de 5 minutes. La superposition de ces deux métriques permet de corréler l'intensité des alertes avec le volume de trafic traité.
— **Distribution des votes** : un histogramme représente la distribution des valeurs du champ `votes`, qui indique le nombre de modèles ayant voté ATTACK pour chaque flux. La majorité des flux présentent un vote de 4/4 (environ 48 000 enregistrements), ce qui témoigne d'un fort consensus entre les modèles sur les cas d'attaques clairs.

8.4 Analyse visuelle en temps réel

L'ensemble des dashboards est configuré en mode rafraîchissement automatique dans Kibana, avec un intervalle de mise à jour de 5 secondes correspondant au trigger du pipeline Spark. Cette synchronisation garantit que les visualisations reflètent en quasi-temps réel l'état du pipeline de détection.

Les filtres interactifs de Kibana permettent aux analystes sécurité de restreindre dynamiquement l'affichage à un protocole, une catégorie d'attaque ou un niveau de sévérité spécifique, sans modifier le dashboard sous-jacent. Le dashboard Kibana est exporté sous forme de fichier `export.ndjson` inclus dans le livrable du projet, permettant de le réimporter en un clic via *Stack Management* → *Saved Objects* → *Import*.

Chapitre 9

Éthique et Conformité

9.1 Protection des données — RGPD

Ce projet traite des données réseau à des fins de détection d'intrusions dans un cadre strictement académique. Les principes fondamentaux du Règlement Général sur la Protection des Données [10] (RGPD — Règlement UE 2016/679) ont été respectés tout au long de la conception et de l'implémentation du système.

Finalité du traitement. Les données utilisées ont pour unique objectif de détecter des anomalies réseau et des menaces de sécurité en temps réel, dans un contexte pédagogique et de recherche académique. Aucune exploitation commerciale n'est envisagée.

Nature des données. Les données traitées proviennent exclusivement de jeux de données publics : le dataset UNSW-NB15, généré en environnement contrôlé par le Cyber Range Lab de l'Université de New South Wales, et des logs SSH OpenSSH structurés. Ces données sont entièrement anonymisées et ne contiennent aucune information personnelle réelle identifiable.

Minimisation des données. Seules les features strictement nécessaires à la détection d'intrusions sont conservées lors du prétraitement. Les colonnes non discriminantes (variance nulle) sont supprimées conformément au principe de minimisation.

Durée de conservation. Les données sont stockées localement de manière temporaire pour les besoins de l'expérimentation. Elles peuvent être supprimées à tout moment sans impact sur les personnes physiques, car aucune donnée personnelle réelle n'est impliquée.

Mesures de sécurité. L'ensemble des services (Kafka, Spark, MongoDB, Elasticsearch, Kibana) est exécuté localement dans un environnement Docker isolé et contrôlé, sans exposition externe.

Droits des personnes. Les jeux de données utilisés ne contenant aucune donnée personnelle réelle identifiable, les droits d'accès, de rectification, d'effacement et de portabilité prévus par le RGPD ne sont pas applicables dans le cadre de ce projet.

9.2 Conformité légale

Le projet s'inscrit dans le respect des cadres réglementaires suivants :

- **RGPD — Règlement (UE) 2016/679** : Règlement du Parlement européen et du Conseil du 27 avril 2016 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel. Les principes de licéité, de limitation des finalités, de minimisation des données, d'exactitude, de limitation de la conservation et d'intégrité/confidentialité sont respectés.
- **Loi marocaine n° 09-08** [11] : Loi relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel, et son décret

d'application n° 2-09-165. Ce projet, réalisé au sein d'un établissement d'enseignement supérieur marocain (ENSA-Tétouan, Université Abdelmalek Essaâdi), est soumis à ce cadre légal national.

— **Commission Nationale de contrôle de la protection des Données à caractère Personnel (CNDP)** : autorité de contrôle marocaine compétente en matière de protection des données. Le traitement réalisé dans ce projet, à des fins exclusivement académiques et sur des données publiques anonymisées, est conforme aux lignes directrices de la CNDP.

Usage éthique. Le système développé est destiné exclusivement à la cybersécurité défensive et à la détection d'intrusions. Il n'est en aucun cas conçu pour mener des attaques informatiques ou faciliter une utilisation malveillante. L'ensemble du code source est documenté et transparent, conformément aux principes de responsabilité et d'explicabilité attendus dans les systèmes d'intelligence artificielle.

Responsable du traitement. Ce projet est réalisé sous la supervision de **Pr. Imad Sassi**, au sein du Département Intelligence Artificielle et Digitalisation de l'ENSA-Tétouan.

Chapitre 10

Conclusion et Perspectives

10.1 Synthèse des résultats

Ce projet a abouti à la conception et à l'implémentation complète d'une plateforme SIEM Big Data temps réel, capable de détecter des intrusions réseau avec une haute précision. Les principaux résultats obtenus sont les suivants :

- **Pipeline de streaming opérationnel** : le pipeline Kafka → Spark → MongoDB/Elasticsearch traite en continu les flux réseau avec une latence de détection inférieure à 5 secondes, grâce au trigger de micro-batch configuré dans Spark Structured Streaming.
- **Performances des modèles IA** : les quatre modèles entraînés sur le dataset UNSW-NB15 atteignent des performances élevées. XGBoost obtient le meilleur compromis global avec un F1-score de 0,9165, un AUC-ROC de 0,9836 et un temps d'inférence de 0,0029 ms par échantillon — le plus rapide des quatre modèles évalués, ce qui le rend particulièrement adapté au traitement en continu.
- **Vote majoritaire robuste** : le mécanisme de vote à seuil configurable ($\theta = 2/4$) réduit les faux positifs isolés tout en maintenant un rappel élevé, essentiel dans un contexte SIEM où manquer une attaque est plus coûteux que générer une fausse alerte.
- **Couverture zero-day** : l'intégration de l'Isolation Forest dans le vote garantit une couverture des attaques inconnues, non présentes dans les données d'entraînement, ce qu'aucun modèle supervisé ne peut assurer seul.
- **Infrastructure reproductible** : le déploiement via Docker Compose permet de démarrer l'ensemble de l'infrastructure (Kafka, Spark, MongoDB, Elasticsearch, Kibana) en une seule commande, sur toute machine disposant de Docker.
- **Conformité éthique** : le projet respecte les principes du RGPD et de la loi marocaine 09-08, en s'appuyant exclusivement sur des données publiques anonymisées dans un cadre académique encadré.

10.2 Limites du système

Plusieurs limites ont été identifiées lors de la réalisation du projet :

- **Données simulées** : le pipeline traite le dataset UNSW-NB15 rejoué depuis un fichier CSV, et non un vrai trafic réseau capturé en direct. Les performances réelles sur un réseau de production pourraient différer en raison de la distribution des données.
- **Absence de traitement du langage naturel (NLP)** : les logs SSH ingérés via le topic `logs-systeme` sont stockés dans MongoDB mais non exploités par les modèles

de détection. En effet, les approches NLP – telles que la classification de séquences de logs par LSTM ou BERT – nécessitent des données textuelles structurées et volumineuses, ce qui n'est pas adapté au dataset UNSW-NB15 retenu, dont les features sont exclusivement numériques et catégorielles. L'intégration du NLP pour l'analyse des logs textuels constitue néanmoins une piste d'amélioration future.

— **Déploiement mono-nœud** : l'architecture Docker Compose est conçue pour un environnement de développement local. Un déploiement en production nécessiterait une infrastructure multi-nœuds (cluster Kafka, cluster Spark, Elasticsearch distribué).

— **Taux de faux positifs de l'Isolation Forest** : en évaluation isolée, l'Isolation Forest génère un taux de faux positifs de 67,14 %, ce qui serait rédhibitoire en production sans le filtrage apporté par le vote majoritaire.

— **Absence d'enrichissement Threat Intelligence** : le topic `threat-intel` a été initialisé mais non alimenté par des flux IoC externes réels. L'intégration de sources de renseignement sur les menaces (listes d'IP malveillantes, signatures STIX/-TAXII) constitue une piste d'amélioration prioritaire.

— **Réentraînement des modèles** : les modèles sont entraînés hors ligne et statiques. Un système de production devrait prévoir un pipeline de réentraînement continu (*continual learning*) pour s'adapter aux nouvelles formes d'attaques.

10.3 Perspectives d'amélioration

Sur la base des limites identifiées, plusieurs axes d'amélioration sont envisagés pour les travaux futurs :

— **Capture de trafic réel** : intégrer un agent de capture réseau (par exemple *Zeek* ou *Suricata*) pour alimenter le topic Kafka avec du trafic réseau réel, en remplacement du rejeu CSV.

— **Analyse NLP des logs textuels** : intégrer un modèle de traitement du langage naturel (BERT, LSTM) pour analyser les logs SSH stockés dans MongoDB, afin d'étendre la détection aux événements textuels en complément des flux réseau numériques.

— **Déploiement distribué** : migrer l'infrastructure Docker Compose vers Kubernetes pour bénéficier d'une scalabilité horizontale, d'une haute disponibilité et d'une gestion automatisée des pannes.

— **Enrichissement Threat Intelligence** : connecter le topic `threat-intel` à des flux IoC publics (AbuseIPDB, OTX AlienVault, MISP) pour enrichir chaque alerte avec des indicateurs de compromission connus.

— **Réentraînement continu** : implémenter un pipeline MLOps (MLflow, Kubeflow) pour le versionnement, l'évaluation et le déploiement automatique de nouvelles versions des modèles au fur et à mesure de l'accumulation de nouvelles données étiquetées.

— **Explicabilité des décisions** : intégrer des méthodes d'explicabilité (SHAP, LIME) pour fournir aux analystes sécurité une justification compréhensible de chaque décision d'alerte, renforçant ainsi la confiance dans le système.

— **Cartographie géographique des menaces** : ajouter une visualisation géographique dans Kibana pour représenter l'origine des adresses IP source des attaques détectées, en exploitant une base de géolocalisation IP (GeoLite2).

— **Alerting automatique** : configurer des règles d'alerte Kibana (ou intégrer ElastAlert) pour notifier automatiquement les équipes de sécurité par email ou webhook lors de la détection d'attaques de sévérité HIGH.

Bibliographie

[1] Moustafa, N. & Slay, J. (2015). *UNSW-NB15 : A Comprehensive Data Set for Network Intrusion Detection Systems*. Military Communications and Information Systems Conference (MilCIS).

[2] Kreps, J., Narkhede, N. & Rao, J. (2011). *Kafka : A Distributed Messaging System for Log Processing*. NetDB Workshop.

[3] Zaharia, M. et al. (2016). *Apache Spark : A Unified Engine for Big Data Processing*. Communications of the ACM, 59(11).

[4] Armbrust, M. et al. (2018). *Structured Streaming : A Declarative API for Real-Time Applications in Apache Spark*. ACM SIGMOD.

[5] Gormley, C. & Tong, Z. (2015). *Elasticsearch : The Definitive Guide*. O'Reilly Media.

[6] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.

[7] Liu, F. T., Ting, K. M. & Zhou, Z. H. (2008). *Isolation Forest*. IEEE International Conference on Data Mining (ICDM).

[8] Chen, T. & Guestrin, C. (2016). *XGBoost : A Scalable Tree Boosting System*. ACM SIGKDD.

[9] Pedregosa, F. et al. (2011). *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research (JMLR), 12, 2825–2830.

[10] Parlement européen et Conseil de l'Union européenne (2016). *Règlement (UE) 2016/679 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (RGPD)*.

[11] Royaume du Maroc (2009). *Loi n° 09-08 relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel*.

Annexe B

Captures d'écran Kibana

Cette annexe présente les captures d'écran des principaux tableaux de bord Kibana réalisés dans le cadre du projet.

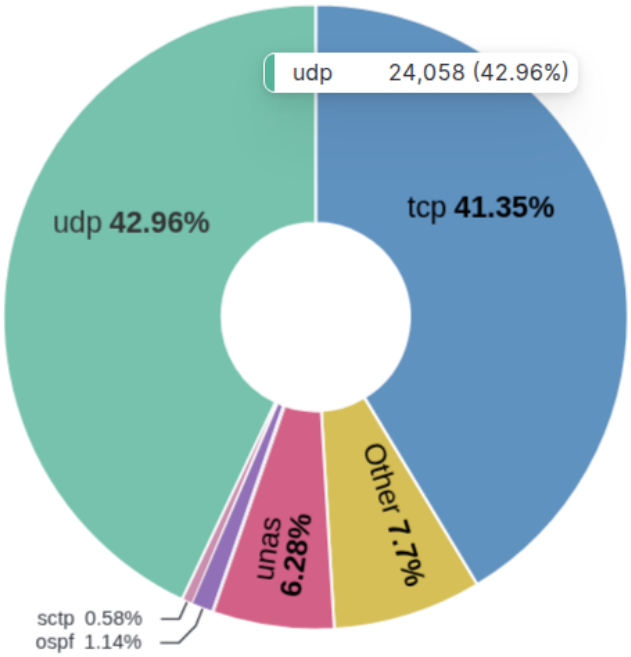


FIGURE B.1 – Répartition des protocoles réseau — graphique en anneau (UDP 42,96 %, TCP 41,35 %)

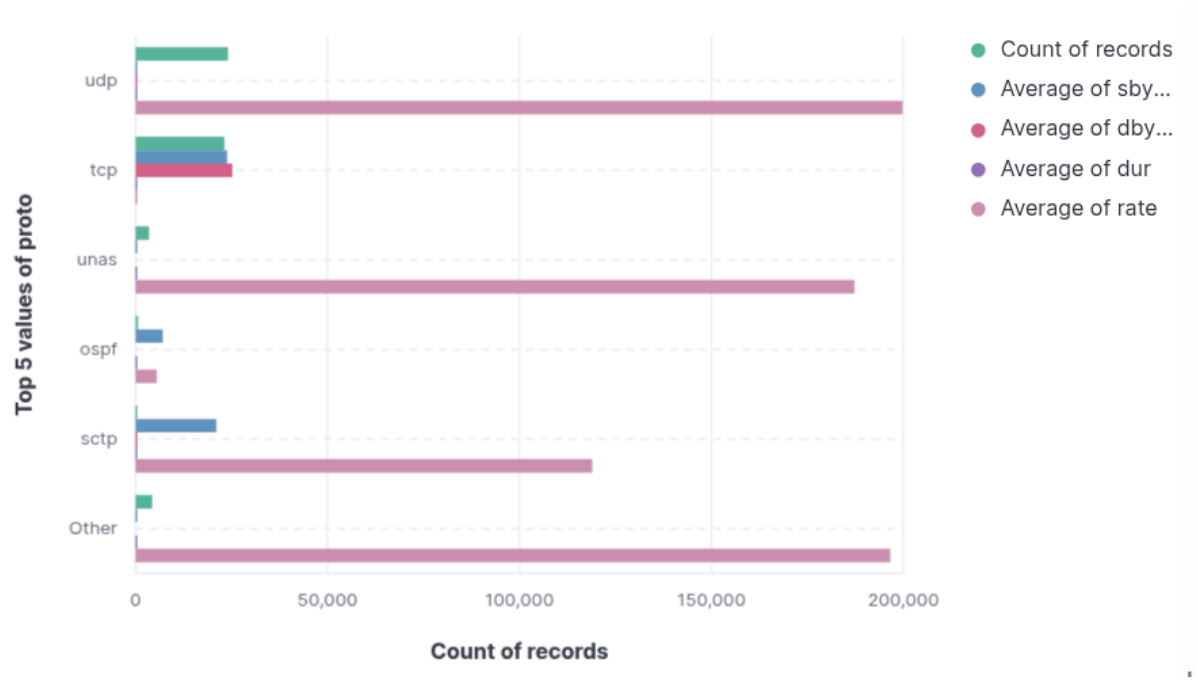


FIGURE B.2 – Distribution des types d'attaques — Generic 33,69 %, Normal 20,02 %, Exploits 19,84 %

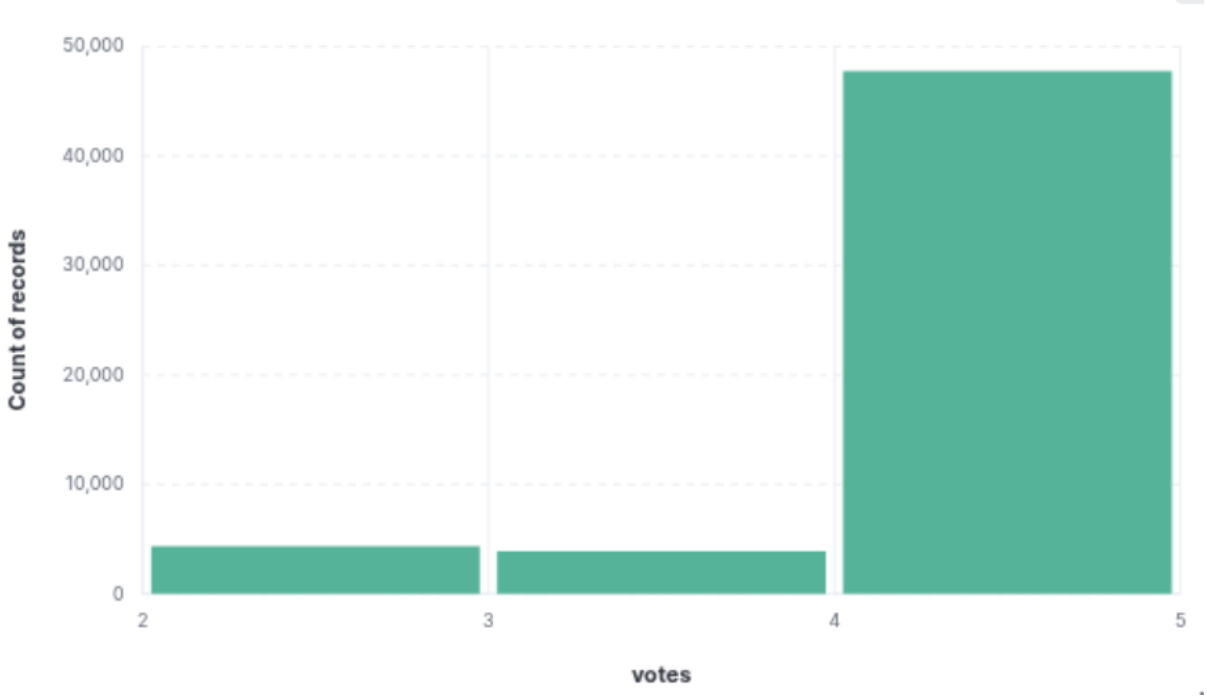


FIGURE B.3 – Nombre de flux par protocole — barres verticales (Top 5)

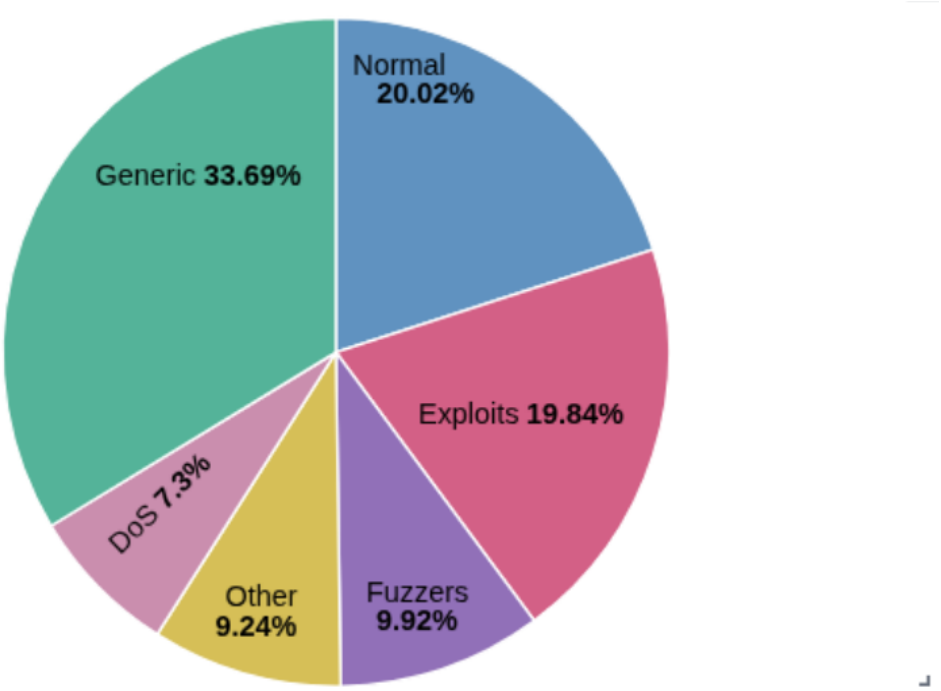


FIGURE B.4 – Métriques moyennes par protocole — octets source/destination, durée et taux de transfert



FIGURE B.5 – Attaques par protocole et catégorie — barres empilées

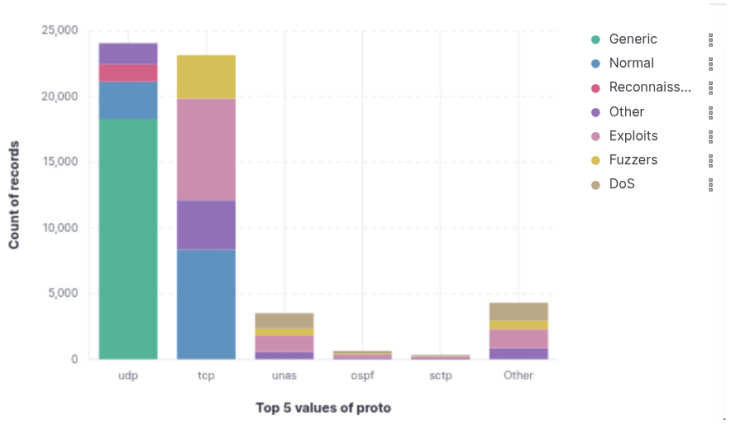


FIGURE B.6 – Score de confiance médian des prédictions (*Median of confidence*)

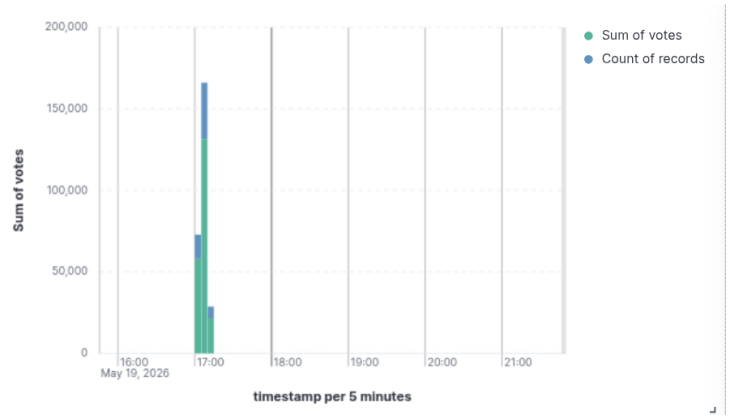


FIGURE B.7 – Dernière valeur du score d'anomalie Isolation Forest (`if_score` $\approx 0{,}4$)



FIGURE B.8 – Évolution temporelle de la somme des votes et du nombre de flux — fenêtres de 5 minutes

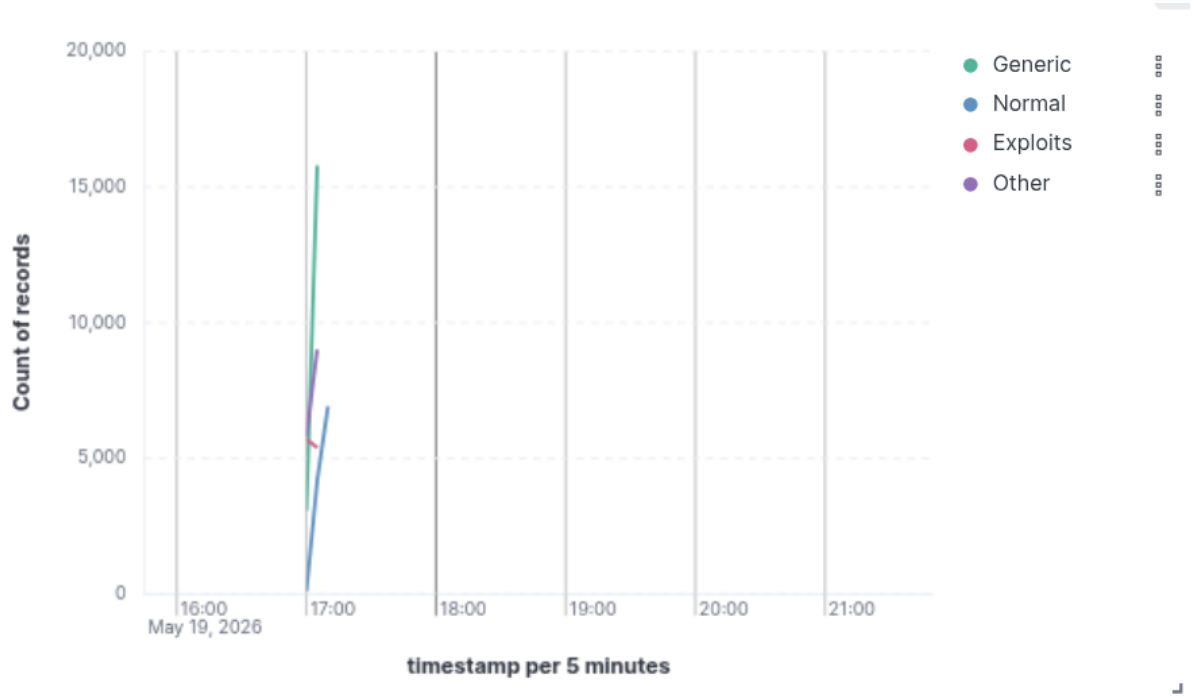


FIGURE B.9 – Évolution temporelle des flux par catégorie d'attaque — fenêtres de 5 minutes

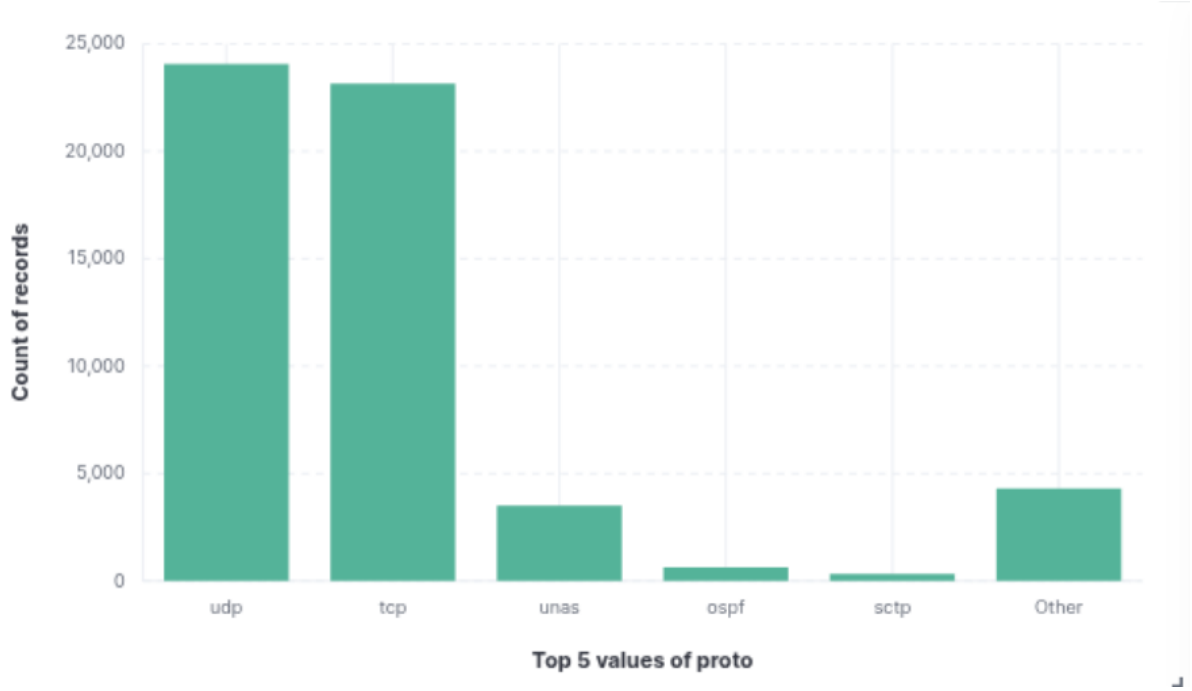


FIGURE B.10 – Distribution des votes ATTACK par flux — majorité à 4/4 modèles (≈48 000 enregistrements)

Annexe C

Configuration Docker Compose

L'ensemble de l'infrastructure du projet est orchestré via un fichier `docker-compose.yml` unique, permettant de démarrer tous les services nécessaires avec la commande `docker compose up -d`. Le fichier complet est présenté ci-dessous.

Service	Image	Port	Rôle
`zookeeper`	`confluentinc/cp-zookeeper:7.4.0`	2181	Coordination Kafka
`kafka`	`confluentinc/cp-kafka:7.4.0`	9092	Message broker
`spark`	`apache/spark:3.5.0`	8080	Moteur de traitement
`mongodb`	`mongo:4.4`	27017	Base documentaire
`elasticsearch`	`docker.elastic.co/elasticsearch:8.13.0`	9200	Moteur d'indexation
`kibana`	`docker.elastic.co/kibana:8.13.0`	5601	Visualisation

TABLE C.1 – Récapitulatif des services Docker Compose

Listing C.1 – Fichier `docker-compose.yml` du projet SIEM

```
services:

  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  spark:
    image: apache/spark:3.5.0
    ports:
      - "8080:8080"
    command: /opt/spark/bin/spark-class org.apache.spark.deploy
       .master.Master
```

```
24
25    mongodb:
26      image: mongo:4.4
27      ports:
28        - "27017:27017"
29
30    elasticsearch:
31      image: docker.elastic.co/elasticsearch/elasticsearch:8.13.0
32      ports:
33        - "9200:9200"
34      environment:
35        - discovery.type=single-node
36        - xpack.security.enabled=false
37
38    kibana:
39      image: docker.elastic.co/kibana/kibana:8.13.0
40      ports:
41        - "5601:5601"
42      depends_on:
43        - elasticsearch
```

Notes de configuration :

- **Zookeeper** ne expose pas de port externe ; il est accessible uniquement en réseau interne Docker par Kafka via `zookeeper:2181`.
- **Kafka** est configuré en mode single-broker avec un facteur de réplication de 1 (`KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1`), adapté à un environnement de développement.
- **Spark** démarre en mode *standalone master* ; les workers Spark et les scripts Python sont lancés séparément via `spark-submit`.
- **Elasticsearch** est configuré en mode nœud unique (`discovery.type: single-node`) avec la sécurité `xpack` désactivée pour simplifier le développement local.
- **Kibana** dépend explicitement d'Elasticsearch (`depends_on: elasticsearch`), garantissant que le moteur d'indexation est opérationnel avant le démarrage de l'interface de visualisation.

Démarrage de l'infrastructure :

```
# D marrer tous les services en arri re-plan
docker compose up -d

# V rifier l' tat  des conteneurs
docker compose ps

# Arr ter tous les services
docker compose down
```